

ESDBox_IPGUI

嵌入式图形系统技术手册

— 绘图引擎与颜色合成系统深度解析 —

模块聚焦：core/gfx（图形渲染）与 core/composite（颜色合成）

适用环境：嵌入式 MCU / 低功耗处理器

语言栈：纯 C99，无运行时依赖

前言

本书立意

本书是一部关于嵌入式图形系统的技术专著。它以 ESDBox_IPGUI 引擎的完整工程实现为蓝本，从最底层的像素着色原理出发，贯穿整个图形渲染管线的建构过程，最终抵达完整控件系统的渲染调度机制。

ESDBox_IPGUI 是一套纯 C 语言实现的嵌入式 2D GUI 引擎，代码总量超过三万行，覆盖从基础内存管理、定点数坐标系统、反走样图元绘制、Alpha 颜色合成到控件树遍历与脏矩形优化的全链路技术栈。本书聚焦其中两个最核心的子系统——图形绘制引擎（gfx）与颜色合成系统（composite）——并对渲染调度层的脏矩形机制与屏幕刷新管线进行完整拆解。

本书的核心立场是：**理解一个图形系统的唯一方法，是追踪从用户指令到屏幕像素的完整数据流**。为此，每一节都从实际源码出发，分析数据结构的设计动机、核心算法的正确性论证、边界条件的处理策略以及性能优化的量化依据。书中引用的所有代码片段均来自项目真实源码，并标注了对应的文件路径和行号。

适用读者

- 具备 C 语言基础、希望系统理解嵌入式图形渲染底层机制的开发者
- 正在自主构建或维护嵌入式 GUI 框架的工程师
- 对实时图形算法（反走样、有符号距离场、三角形光栅化、多边形活动边表）有研究兴趣的学者
- 需要从完整工程实践中提取架构设计参考的系统架构师

本书不要求读者具备计算机图形学的预备知识——第一章会从零开始建立坐标系统和反走样的直觉。但读者需要对 C 语言的结构体、联合体、指针运算和位操作有基本的熟悉。

阅读建议

本书共分两章，采用递进式结构。

第一章建立了"子像素坐标系"和"遮罩数组"两个核心概念，它们是理解整个渲染管线的基础。如果你跳过了第一章直奔第二章，会遇到大量无法理解的前置引用。

每节内容在逻辑上是自包含的。你可以将任一节作为独立的技术参考文档查阅，但理解整个系统的设计动机，仍然需要跟随全书的脉络。文中大量交叉引用了前文的术语和概念，建议首读时不要跳节。

在每章末尾，我附上了"重点知识回顾"和"思考问题"。前者是帮你快速梳理核心概念的索引，后者是帮助你验证理解深度的自测工具。如果你能独立回答所有思考问题，说明你已经掌握该章的核心思想。

工程结构导航

为保证叙述的准确性，全书引用的代码均来自 ESDBox_IPGUI 工程的实际文件。以下是本书涉及的核心模块的目录结构：

```
core/gfx/  
├─ ipgui_edge_halfplane_mask.c/h  
├─ ipgui_edge_wdf_mask.c/h  
├─ ipgui_ring_mask.c/h  
├─ ipgui_mask_gradient.c/h  
├─ ipgui_mask_buf.c/h  
├─ ipgui_image_buf.c/h  
├─ ipgui_image_mask.c/h  
├─ ipgui_draw_line.c/h  
├─ ipgui_draw_arc.c/h  
├─ ipgui_draw_triangle.c/h  
├─ ipgui_draw_polygon.c/h  
├─ ipgui_draw_pixel.c/h  
├─ ipgui_draw_box_background.c/h  
├─ ipgui_draw_box_border.c/h  
├─ ipgui_draw_box_shadow.c/h
```

```
core/composite/  
├─ ipgui_blend.c/h
```

- 第一章：图形绘制引擎
- 半平面边缘遮罩（Q26.6 定点数反走样）
- WDF 宽线分布函数遮罩
- 环形遮罩（圆角矩形 SDF + LRU 缓存）
- 遮罩渐变
- 遮罩缓冲区管理
- 图像缓冲区管理
- 图像遮罩生成
- 线段绘制（Wu 经典细线 + 宽线 + 线帽）
- 弧/圆/扇形/圆环统一接口
- 三角形光栅化（半平面遮罩 + 三点排序法）
- 多边形光栅化（扫描线 + AVL 活动边表）
- 单像素混合绘制
- Box 背景（圆角矩形分解策略）
- Box 边框（四条边 + 四个圆角）
- Box 阴影（SDF + 1D 多项式 CDF 近似）

- 第二章：颜色合成系统
- Paint 分发器

└─ blend_color/ipgui_blend_color.c/h	— 纯色混合（函数表 + packed blend）
└─ blend_gradient/ipgui_blend_gradient_color.c/h	— 渐变混合（方向优化快速通道）
└─ blend_image/ipgui_blend_image.c/h	— 图像混合（格式交叉矩阵）
core/ui/widget_manager/	— 渲染调度层
└─ ipgui_dirty_rect.c/h	— 脏矩形管理器（DFS 收集 + 相邻合并）
└─ ipgui_widget_tree.c/h	— 控件树遍历
al/hal/	— 硬件抽象 + 渲染入口
└─ ipgui_screen.c/h	— 屏幕渲染主入口（PFB 分配 + 逐片渲染）
└─ ipgui_rect_slice.c/h	— 矩形切片器（贪心列优先 + 除法消除）
base/inc/	— 基础设施
└─ ipgui_memory.h	— 内存管理（内存池 + 多策略分配器）
└─ ipgui_avl.h	— AVL 自平衡二叉搜索树
└─ ipgui_membox.h	— 固定大小内存块分配器
└─ ipgui_list.h	— 双向链表（内核风格 list_head）

排版约定

全书遵循以下排版规则：

- 常量/宏：IPGUI_PIXEL_BITS、EDGE_EMBED_NUM（全大写+下划线）
- 函数名：ipgui_blend()、ipgui_edge_wdf_mask()（小写+下划线，前缀 ipgui_）
- 类型名：ipgui_color_t、ipgui_surf_t（小写+下划线，后缀 _t）
- 文件路径：core/gfx/ipgui_draw_line.c（使用等宽字体 Markdown 包裹）
- 代码片段：所有源码均来自真实工程文件，行号与源文件保持一致

术语表

术语	缩写	定义
遮罩（Mask）	mask	0-255 的 u8 数组，控制颜色在像素级的作用强度
表面（Surface）	surf	描述一段像素缓冲区的元信息（坐标范围、跨度、像素格

		式)
脏矩形 (Dirty rect)	DR	屏幕上需要重绘的矩形区域
部分帧缓冲 (Partial FB)	PFB	因内存限制而只覆盖脏矩形的临时像素缓冲区
预乘 Alpha	PM Alpha	颜色通道在混合前已与 Alpha 通道相乘的存储格式
Q26.6 定点数	-	32 位整数, 高 26 位整数部 / 低 6 位小数部 (精度 1/64)
有符号距离场	SDF	每个像素存储到最近形状边界的带符号距离
活动边表	AET	扫描线算法中维护当前扫描线与多边形相交边的数据结构
包容性填充规则	Non-zero / Even-odd	多边形填充的两种判定规则
半平面 (Halfplane)	-	二维平面被一条有向直线分割成的两个区域

第一章 嵌入式绘图引擎核心架构与实现

****模块位置****: ``core/gfx/``

****核心职责****: 几何遮罩生成、子像素反走样、图元光栅化

****对外接口****: 遮罩数组 (`mask`) + 涂料描述 (`paint`) → 委托 `composite` 模块完成像素混合

1.1 架构概览

1.1.1 模块定位与职责边界

ESDBox_IPGUI 的渲染管线采用严格的分层架构。`core/gfx` 模块位于管线的中间层：它向上接收控件的绘制请求，向下将计算结果传递给 `core/composite` 模块。两层之间的接口由两样东西定义——**遮罩数组** (`mask`) 与 **涂料描述** (`paint`)。

`gfx` 模块不关心颜色。它的全部职责是处理几何问题：一条直线的斜率如何影响像素覆盖度、一个圆角矩形在任意象限的 SDF 如何计算、三角形三个顶点按 `y` 坐标排序后如何拆分为上下两个子三角形的光栅化范围、多边形的边表如何按扫描线递增维护。

`composite` 模块不关心形状。它接收一个 `u8_t` 数组 (`mask`) 和一个 `ipgui_paint_t` 结构体，然后逐像素地将颜色混入表面。它不知道这个 `mask` 来自直线还是圆角——它只知道"在这个像素上，遮罩值是 128"。

这种分工带来了关键的架构收益：

1. **新增图元不影响 `composite`**。今天给 `gfx` 增加五角星的绘制逻辑，`composite` 的所有混合代码无需修改。

2. **新增颜料类型不影响 gfx**。明天给 composite 增加锥形渐变，gfx 的所有图元绘制代码无需修改。

3. **两个模块可以独立测试与优化**。gfx 的优化方向是减少几何计算量；composite 的优化方向是减少颜色混合的算术运算次数。

以绘制一个蓝色圆角按钮为例，数据在两模块之间的流动如下：

```
用户代码: ipgui_draw_box_background(surf, clip, &abs, &shape, &bg_style)
|
|— 步骤 1: 计算 padding_box (考虑 padding 的内缩矩形)
|— 步骤 2: 规整圆角半径 (get_max_radius)
|— 步骤 3: 将矩形分解为 5 个不含圆角的矩形条 + 4 个圆角区域
|— 步骤 4: 对每个圆角区域, 调用环形遮罩模块 (ipgui_ring_mask)
|
|— 步骤 5: 对每个子区域调用 ipgui_blend(surf, clip, dest, paint, opacity,
mask, ...)
    → composite 模块接管: 遮罩 × paint 颜色 × 目标像素 = 最终结果
```

步骤 1-4 是纯粹的几何运算，完全不涉及颜色。步骤 5 是颜色运算，完全不涉及几何。

这种清晰的边界是 ESDBox_IPGUI 管线设计的核心原则。

1.1.2 核心数据类型体系

gfx 模块定义了数种贯穿所有子模块的基础类型。理解这些类型是理解后续算法的前提。

```
/* — 源文件: core/gfx/ipgui_core.h — */

/* 像素表面描述符 */
typedef struct {
    ipgui_aabb_t    surf;        /* 表面在屏幕上的全局坐标范围 */
    u8_t            * color;     /* 指向 surf 区域首像素的内存指针 */
    u32_t           stride;      /* 每行字节跨度 (可大于 wxpix_size) */
    ipgui_pix_fmt_t pix_fmt;     /* 像素格式枚举 */
    u8_t            pix_size;    /* 每像素字节数 */
} ipgui_surf_t;

/* 轴对齐包围盒 */
typedef struct {
    ipgui_coord_t start_x;
    ipgui_coord_t start_y;
    ipgui_coord_t end_x;
    ipgui_coord_t end_y;
}
```

```
} ipgui_aabb_t;
```

```
/* 二维点 */
```

```
typedef struct {  
    ipgui_coord_t x;  
    ipgui_coord_t y;  
} ipgui_point_t;
```

`ipgui_surf_t` 是整个绘制操作的"画布"描述符。关键设计点在于 `stride` 字段——它允许行字节跨度大于 `宽度 × 每像素字节数`。当屏幕驱动使用部分帧缓冲（PFB）时，分配的缓冲区宽度可能大于脏矩形宽度（以满足对齐要求），此时 `stride` 保证逐行寻址的准确性。

另一个重要细节是 `surf` 字段的类型为 `ipgui_aabb_t`，其中存储的是**屏幕全局坐标**而非局部相对坐标。这意味着对像素数组 `color` 的任何偏移寻址都必须先用 `x - surf.start.x` 将屏幕坐标转换为表面局部坐标——这是全模块反复出现的模式。

```
/* — 源文件: core/gfx/ipgui_color.h — */
```

```
typedef union {  
    struct { u8_t b, g, r, a; }; /* 小端序内存布局: [B][G][R][A] */  
    u32_t v; /* 整体 32 位值 */  
} ipgui_color_t;
```

union 设计是嵌入式图形编程中的一个经典技巧。它允许代码根据场景需求选择最优的访问方式：

- **结构化通道访问**: `color.r`、`color.g`、`color.b`、`color.a`——适合算法描述和单通道操作

- **批量 32 位操作**: `color.v`——适合全通道等值比较（`if (color.v == 0)` 检查是否全透明）和颜色拷贝赋值

这种设计在无需依赖结构体布局假设的前提下，既保留了代码可读性，又获得了位级优化的能力。注意注释中明确注明了 `struct { b, g, r, a }` 的成员顺序依赖于小端序——这是嵌入式平台上的默认字节序假设，移植到大端序平台时需调整成员顺序。

1.1.3 gfx 模块的完整文件清单

```
core/gfx/
```

— ipgui_edge_halfplane_mask.c/h	— 半平面边缘遮罩：反走样的基础单元
— ipgui_edge_wdf_mask.c/h	— WDF 宽线厚度遮罩：宽线段的覆盖度渐变
— ipgui_ring_mask.c/h	— 环形遮罩：圆角矩形 SDF + LRU 缓存
— ipgui_mask_gradient.c/h	— 遮罩渐变：遮罩值沿线性/径向渐变分布
— ipgui_mask_buf.c/h	— 遮罩缓冲区：临时遮罩内存的获取与释放
— ipgui_image_buf.c/h	— 图像缓冲区：图像像素内存的获取与释放
— ipgui_image_mask.c/h	— 图像遮罩：从图像数据生成遮罩值
— ipgui_draw_line.c/h	— 线段绘制：Wu 经典细线 + 宽线 + 线帽
— ipgui_draw_arc.c/h	— 弧/圆/扇形/圆环统一接口
— ipgui_draw_triangle.c/h	— 三角形光栅化：半平面遮罩 + 三点排序法
— ipgui_draw_polygon.c/h	— 多边形光栅化：扫描线 + AVL 活动边表
— ipgui_draw_pixel.c/h	— 像素绘制：单像素混合
— ipgui_draw_box_background.c/h	— Box 背景：圆角矩形分解策略
— ipgui_draw_box_border.c/h	— Box 边框：四条边 + 四个圆角
— ipgui_draw_box_shadow.c/h	— Box 阴影：SDF + 1D 多项式模糊

这十五个模块中，最底层的是半平面边缘遮罩（`edge_halfplane_mask`）——它是三角形、弧、多边形等所有"形状边界"遮罩的计算单元。环形遮罩（`ring_mask`）是圆角矩形场景的专业优化。WDF 宽线遮罩是线段加粗的专用处理模块。其余的 `draw_*` 模块将遮罩和 `composite` 混合组合成可调用的公开 API。

1.2 子像素坐标系统

1.2.1 离散像素网格的固有局限

计算机屏幕由离散的像素网格构成。当一条数学上的理想线段穿过屏幕时，只有极少数像素的中心恰好落在线段上——大多数像素的中心与线段之间有一个不等的垂直距离。如果仅用整数坐标（第几个像素）描述图形，每个像素只有"画"和"不画"两种状态，结果必然是阶梯状的锯齿。

反走样（`anti-aliasing`）的核心思路是将每个像素的"不透明度"与其到图形边界的距离关联：距离为零的像素完全不透明（强度 255），距离逐渐增加时强度逐渐降低（0~255），超过一个像素宽度的像素完全透明（强度 0）。

实现这一思路需要比整数像素更细粒度的坐标系统。

1.2.2 Q26.6 定点数格式

ESDBox_IPGUI 使用 32 位有符号整数作为子像素坐标系的基础类型，采用 Q26.6 定点数格式：

```
/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.h:17 — */  
typedef s32_t ipgui_edge_coord_t; /* Q26.6: 高 26 位为整数部，低 6 位为小数部 */
```

Q26.6 格式将每个整数像素等分为 $2^6 = 64$ 个子像素单元。选择 64 而非 256（Q24.8）或 16（Q28.4）的工程理由是：

1. **乘法效率**：乘以 64 等价于 $\ll 6$ ，除以 64 等价于 $\gg 6$ ，全部用位移操作实现，不依赖硬件乘法器或除法器。
2. **精度匹配**：64 级离散度在 8 位颜色深度（256 级透明度）下误差不超过 ± 1 级——人眼无法分辨此差异。
3. **整数范围**：Q26.6 可表示的坐标范围为 $\pm 33,554,431$ 像素，远超任何实际嵌入式屏幕的尺寸（通常 $\leq 1024 \times 768$ ）。

一个典型的 320×240 屏幕在子像素空间中的分辨率等价于 20480×15360 。一条从 (2.3, 1.1) 到 (7.8, 5.6) 的线段在子像素空间中表示为从 $2 \times 64 + 19 = 147$ 到 $7 \times 64 + 51 = 499$ 的整数坐标。

1.2.3 坐标对齐工具函数

四个基础工具函数构成整数坐标与子像素坐标的转换桥梁：

```
/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:107-119 — */  
  
/* 返回 c 的低 6 位——即 c 到其下方最近整数像素中心的子像素距离 */  
ipgui_edge_coord_t dist_to_lower_64(ipgui_edge_coord_t c)  
{  
    return c & 63; /* 取低 6 位，等价于 c % 64 */  
}  
  
/* 返回 c 到其上方最近整数像素中心的子像素距离 */  
ipgui_edge_coord_t dist_to_upper_64(ipgui_edge_coord_t c)  
{  
    return (64 - (c & 63)) & 63;  
    /*          ^^^^^^^^^      ^^^^^
```

```

        *           到上边界的距离    边界修正: c 恰好对齐时 64 & 63 = 0 */
    }

    /* 向下对齐到 64 的倍数 (等价于 (c/64)*64) */
    ipgui_edge_coord_t align_down_64(ipgui_edge_coord_t c)
    {
        return c & ~(ipgui_edge_coord_t)63; /* 清零低 6 位 */
    }

    /* 向上对齐到 64 的倍数 (等价于 ceil(c/64)*64) */
    ipgui_edge_coord_t align_up_64(ipgui_edge_coord_t c)
    {
        return (c + 63) & ~(ipgui_edge_coord_t)63;
    }

```

`dist_to_upper_64` 中的 `& 63` 修正是一个值得专门讨论的边界处理细节。当 `c & 63 == 0` (即 `c` 恰好是 64 的整数倍) 时, `64 - 0 = 64`。但一个恰好落在整数像素中心的点, 它到"下一个像素中心"的距离应该是 0 而非 64。`64 & 63 = 0` 修正了这一边界。这一行代码用零分支消除了一个显式的 if-else, 而分支预测失败在 Cortex-M 处理器上通常消耗 2-3 个周期——在被数百条扫描线重复调用的上下文中, 这些周期会快速累积。

1.3 半平面边缘遮罩

1.3.1 边参数化数据结构

半平面边缘遮罩模块是 gfx 层最底层的计算单元。三角形光栅化、弧的扇形裁剪、多边形的边表生成、宽线的覆盖度计算——所有这些高层功能最终都委托给此模块执行"已知一条有向线段, 求某像素到线段的有符号距离"这一原子操作。

每条边缘被参数化为一个紧凑的结构体:

```

/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.h:24-32 — */
typedef struct {
    ipgui_edge_coord_t x1;          /* 线段端点中 y 较小的端点 */
    ipgui_edge_coord_t y1;
    ipgui_edge_coord_t dx;         /* Δx (保留符号) */
    ipgui_edge_coord_t dy;         /* Δy (恒为非负, 因为 y1 已经选了较小的端点) */
}

```

```

s32_t      delta_x; /* 保留字段，实际未使用 */
s32_t      delta_y; /* 平坦边垂直步进因子 (dy/dx 的 Q16.16 表示) */
u8_t       correction_frac_index : 7; /* 修正因子表索引 (0~126) */
u8_t       flatten : 1; /* 0=陡峭边, 1=平坦边 (|dx| >=
dy) */
} ipgui_edge_param_t;

```

两个关键设计细节：

1. **`flatten` 位域**（1 比特）：将边分为"平坦边"（ $|dx| \geq dy$ ）和"陡峭边"（ $|dx| < dy$ ）。分类的意义在于距离计算方式不同——平坦边沿水平方向扫描时，相邻像素的水平增量需要乘以斜率（ dy/dx ）才能转换为垂直距离；陡峭边沿垂直方向扫描时，水平增量直接就是垂直距离。1 比特的存储选择反映了嵌入式开发中"抠门到极致"的内存哲学。

2. **`correction\_frac\_index` 位域**（7 比特）：存储修正因子在全局查找表中的索引。当边不是精确 45° 时，按扫描方向算出的"轴向距离"与"真正的垂直距离"之间存在一个固定的比例偏差。修正因子表预先计算了 127 个离散斜率对应这个偏差，避免了运行时的浮点三角函数计算。

1.3.2 边缘参数初始化

初始化函数的核心任务是：保证 $y1$ 是 y 较小的端点（使 dy 非负）、根据斜率分类、预计算 δy （平坦边的垂直步进因子）和修正因子索引。

```

/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:41-85 — */
ipgui_edge_param_t ipgui_edge_param_init(
    ipgui_edge_coord_t x1, ipgui_edge_coord_t y1,
    ipgui_edge_coord_t x2, ipgui_edge_coord_t y2)
{
    ipgui_edge_param_t param;
    ipgui_memset(&param, 0, sizeof(ipgui_edge_param_t));

    /* 步骤 1: 确保 y1 ≤ y2, 保证 dy 非负 */
    if (y2 > y1) {
        param.dy = y2 - y1;
        param.dx = x2 - x1;
        param.x1 = x1;
        param.y1 = y1;
    } else {
        param.dy = y1 - y2;
        param.dx = x1 - x2;
    }
}

```

```

    param.x1 = x2;
    param.y1 = y2;
}

/* 步骤 2: 斜率分类 */
if (IPGUI_ABS(param.dx) >= param.dy) {
    param.flatten = 1; /* 平坦边 */

    /* 预计算 delta_y = dy/dx (Q16.16 格式) */
    s64_t scaled_dy = (s64_t)param.dy << 16;
    s32_t half_dx = param.dx / 2;

    if (param.dx > 0) {
        param.delta_y = (scaled_dy + half_dx) / IPGUI_ABS(param.dx);
    } else if (param.dx < 0) {
        param.delta_y = (scaled_dy - half_dx) / IPGUI_ABS(param.dx);
    }

    /* 计算修正因子索引: dy/dx 的归一化映射 */
    s32_t abs_dx = IPGUI_ABS(param.dx);
    s32_t abs_dy = param.dy;
    param.correction_frac_index = (u8_t)(
        (abs_dy * 126 + (abs_dx >> 1)) / abs_dx);
} else {
    param.flatten = 0; /* 陡峭边 */

    s32_t abs_dx = IPGUI_ABS(param.dx);
    s32_t abs_dy = param.dy;
    param.correction_frac_index = (u8_t)(
        (abs_dx * 126 + (abs_dy >> 1)) / abs_dy);
}

return param;
}

```

关于 `delta_y` 的计算需要特别说明。`scaled_dy = (s64_t)param.dy << 16` 将 `dy` 放到 Q16.16 格式（低 16 位为小数部）。`+ half_dx` 实现向最近整数的舍入（而非向下取整），这是 $(scaled_dy + half_dx) / abs_dx$ 的等效形式。注意根据 `dx` 的符号分别处理 `+ half_dx` 和 `- half_dx`——这是含符号分母整数除法的舍入方向修正。

关于 $(abs_dy * 126 + (abs_dx \gg 1)) / abs_dx$ ：126 是修正因子表的容量上限（127 项，索引 0~126）。当斜率为 1（精确 45°）时， $abs_dy / abs_dx = 1$ ，计算得 $(126 + 0) = 126$ ，即取索引 126。当斜率接近 0（近乎水平）时， $abs_dy /$

$abs_dx \approx 0$ ，索引接近 0。这种线性映射基于一个工程事实：修正因子在不同斜率下近似线性变化。

1.3.3 扫描线描述符

对于给定的扫描线 y ，边缘在该行上的遮罩生成信息被打包为描述符：

```
/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.h:35-40 — */
typedef struct {
    edge_halfplane_dir_t dir;          /* LEFT 或 RIGHT, 决定遮罩在半平面的哪一侧生效 */
    ipgui_coord_t y;                  /* 当前扫描线的 y 坐标 */
    ipgui_coord_t x_start;            /* 第一个需要生成遮罩的像素的 x 坐标 (整数像素) */
    ipgui_edge_coord_t frac_x;        /* x_start 像素到边界的子像素距离 */
    ipgui_edge_param_t * p;           /* 指向边缘参数结构体的指针 */
} ipgui_edge_halfplane_mask_dsc_t;
```

生成描述符的函数 `ipgui_gen_edge_halfplane_mask_dsc` 针对给定的扫描线 y 计算出边界穿过的精确 x 坐标 (子像素精度)，然后确定"从哪个整数像素开始需要遮罩"：

```
/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:123-136 — */
void ipgui_gen_edge_halfplane_mask_dsc(
    ipgui_edge_halfplane_mask_dsc_t * res,
    edge_halfplane_dir_t dir,
    ipgui_edge_param_t * p,
    ipgui_coord_t y)
{
    ipgui_edge_coord_t x_sub = edge_x_at_y(p, y); /* 已知 y, 求边界 x 坐标 (子像素精度) */

    res->dir = dir;
    res->p = p;
    res->y = y;

    if (dir == EDGE_HALFPLANE_DIR_LEFT) {
        /* 左半平面: 从 x_sub 向上对齐到下一个整数像素开始遮罩 */
        res->x_start = align_up_64(x_sub) >> 6; /* 除以 64 转换为整数像素坐标 */
        res->frac_x = dist_to_upper_64(x_sub);
    } else {
        /* 右半平面: 从 x_sub 向下对齐到当前整数像素开始遮罩 */
        res->x_start = align_down_64(x_sub) >> 6;
        res->frac_x = dist_to_lower_64(x_sub);
    }
}
```

`edge_x_at_y` 函数通过斜率关系计算给定 y 处的边界 x 坐标：

```

/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:7-14 — */
ipgui_edge_coord_t edge_x_at_y(ipgui_edge_param_t * p, ipgui_coord_t y)
{
    if (p->dy == 0) return 0; /* 水平线的边界处理（实际由上层保证不使用此函数） */

    s64_t temp;
    ipgui_edge_coord_t dy = (y * 64) - p->y1; /* 子像素 y 偏移 */
    temp = (s64_t)dy * p->dx;
    return temp / p->dy + p->x1;
}

```

关键点: $(y * 64)$ 将整数像素 y 转换为子像素 y 坐标, 保证与 $p \rightarrow y1$ (子像素坐标) 的单位一致。 $(s64_t)dy * p \rightarrow dx$ 使用 64 位中间量防止溢出 (dy 和 dx 都是子像素坐标, 乘积可能超出 32 位范围)。 $temp / p \rightarrow dy$ 是带舍入方向的整数除法 (向零舍入)。

1.3.4 逐像素遮罩查询

`ipgui_edge_halfplane_mask` 是最终的遮罩查询入口: 给定描述符和像素的 x 坐标, 返回 0-255 的遮罩值:

```

/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:143-162 — */
u8_t ipgui_edge_halfplane_mask(
    ipgui_edge_halfplane_mask_dsc_t * dsc,
    ipgui_edge_coord_t x)
{
    ipgui_edge_coord_t d;
    ipgui_edge_coord_t delta_x;

    /* 确定方向并计算 delta_x */
    if (dsc->dir == EDGE_HALFPLANE_DIR_LEFT) {
        if (x < dsc->x_start) return 255; /* 完全在左半平面内, 完全不透明 */
        delta_x = ((x - dsc->x_start) << 6) + dsc->frac_x;
    } else {
        if (x > dsc->x_start) return 255; /* 完全在右半平面内, 完全不透明 */
        delta_x = ((dsc->x_start - x) << 6) + dsc->frac_x;
    }

    /* 平坦边: 将水平距离转换为垂直距离 */
    if (dsc->p->flatten) {
        d = ((s64_t)delta_x * dsc->p->delta_y + 32768) >> 16;
    } else {
        d = delta_x; /* 陡峭边: 水平距离直接就是垂直距离 */
    }
}

```

```

/* 应用修正因子并映射到 0~255 */
correct_d_and_return_mask(d);
}

```

这里有两个核心数学操作需要详细解释。

delta_x 的计算: $((x - \text{dsc->x_start}) \ll 6) + \text{dsc->frac_x}$ 。 x 和 x_start 是整数像素坐标，它们的差乘以 64 转换为子像素距离。加上 frac_x (x_start 到边界本身的子像素偏移)，得到当前像素到边界的完整子像素距离。

平坦边的距离转换: $((\text{s64_t})\text{delta_x} * \text{dsc->p->delta_y} + 32768) \gg 16$ 。

delta_y 是 dy/dx 的 Q16.16 表示。当扫描线沿水平方向前进一个像素宽度 (64 个子像素) 时，边界线在垂直方向上的位移是 $64 \times dy/dx$ 个子像素。 $\text{delta_x} \times \text{delta_y}$ 就是当前像素的水平距离对应的垂直距离 (仍为 Q16 精度)。 $+ 32768$ (即 0.5×65536) 实现向最近整数舍入。 $\gg 16$ 将结果还原为整数子像素距离。

1.3.5 修正因子宏

`correct_d_and_return_mask` 宏完成距离修正和遮罩值映射:

```

#define correct_d_and_return_mask(distance)\
    distance = distance * correction_frac[\text{dsc->p->correction\_frac\_index}] \gg 8;\
    if (!distance) return 255;\
    else if (distance > 64) return 0;\
    else return (64 - distance) << 2;

```

修正因子的分母为 256 (通过 $\gg 8$ 实现)。取索引对应斜率下的真实垂直距离 = 轴向距离 $\times$ 修正因子 / 256。

距离映射规则:

- $\text{distance} == 0$: 像素中心恰好在边界上 $\rightarrow$ 遮罩值 255 (完全不透明)
- $\text{distance} > 64$: 像素中心距离超过一个整像素宽度 $\rightarrow$ 遮罩值 0 (完全透明)
- $0 < \text{distance} \leq 64$: 线性过渡 $\rightarrow$ 遮罩值 $= (64 - \text{distance}) \times 4 = 256 - 4 \times \text{distance}$

$\ll 2$ (乘以 4) 将 0~64 的范围线性映射到 0~255: 当 $\text{distance}=0$ 时 $64 \ll 2 = 256$, $\text{distance}=64$ 时 $0 \ll 2 = 0$ 。注意当 $\text{distance}=0$ 时已经在第一个分支返回了 255, 因此 $(64 - \text{distance}) \ll 2$ 不会输出 256 (超出 u8 范围)。

1.3.6 修正因子表

```
/* — 源文件: core/gfx/ipgui_edge_halfplane_mask.c:20-27 — */
const u8_t correction_frac[127] = {
    255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 255, 254, 254,
    254, 254, 254, 253, 253, 253, 253, 252, 252, 252, 251, 251, 251, 250, 250,
    250, 249, 249, 248, 248, 247, 247, 247, 246, 246, 245, 245, 244, 244, 243,
    /* ... 共 127 个值 ... */
    194, 193, 192, 192, 191, 190, 189, 189, 188, 187, 186, 186, 185, 184, 183,
    183
};
```

此表的数学含义：对于斜率为 $\text{abs}(\text{dy})/\text{abs}(\text{dx})$ 的边，其"轴向扫描距离"与"真实垂直距离"的比例为 $\text{sqrt}(\text{dx}^2 + \text{dy}^2) / |\text{dx}|$ （勾股定理）。修正因子 = $256 \times |\text{dx}| / \text{sqrt}(\text{dx}^2 + \text{dy}^2)$ 。例如， 45° 时 $|\text{dx}| = |\text{dy}|$ ，修正因子 = $256 / \text{sqrt}(2) \approx 181$ 。表的前 14 个值全部为 255，表明当斜率 $< 1/10$ 时，轴向距离与垂直距离的偏差小于 $1/255$ ——肉眼无法分辨，直接取 255 即可。

127 字节的查找表在 64KB RAM 的嵌入式系统中占 0.2% 的内存。这是一个"经过深思熟虑的奢侈"——它换来了运行时零浮点计算和零除法的反走样效果。

1.4 环形遮罩——圆角矩形的 SDF 引擎

1.4.1 设计动机

圆角矩形是 GUI 中最频繁出现的形状——按钮、输入框、卡片、面板，几乎全部基于圆角矩形。如果每个圆角矩形都在运行时用半平面遮罩或其他通用方法逐像素计算，性能将不可接受。

环形遮罩（ring mask）模块专门为圆角矩形场景做了极致的优化。它使用有符号距离场（SDF）预先计算每个像素到圆角边缘的距离，并将计算结果缓存在 LRU 链表中以复用。

1.4.2 圆角参数与初值估计

```
/* — 源文件: core/gfx/ipgui_ring_mask.c — */
typedef struct {
    ipgui_coord_t r;          /* 圆角半径 */
    s32_t         r2;         /*  $r^2$  (避免运行时乘法) */
    s32_t         rmax2;      /*  $(r+1)^2$  — 遮罩外边界 */
    s32_t         rmin2;      /*  $(r-1)^2$  — 遮罩内边界 */
    s32_t         inv_out;    /* 外过渡区反比系数 */
    s32_t         inv_in;     /* 内过渡区反比系数 */
} corner_param_t;

void out_corner_param_init(corner_param_t * p, ipgui_coord_t r)
{
    p->r      = r;
    p->r2     = r * r;
    p->rmax2  = (r + 1) * (r + 1);
    p->rmin2  = (r - 1) * (r - 1);
    p->inv_out = (0xffff) / (p->rmax2 - p->r2);
    p->inv_in  = (0xffff) / (p->r2 - p->rmin2);
}
```

遮罩的过渡区域限定在 $[r-1, r+1]$ 范围内 (± 1 像素宽度)。在此范围内, 距离通过 `inv_out` 和 `inv_in` 的倒数线性插值转换为 0~255 的遮罩值。此设计的本质是 **smoothstep 的线性近似**——在反走样所需的 1 像素过渡带内, 线性插值在 8 位颜色深度下与 `smoothstep` 的视觉差异人眼无法分辨, 而避免了运行时的三次多项式计算。

1.4.3 LRU 缓存机制

环形遮罩的计算结果被缓存为 `corner_mask_cache_item_t`:

```
/* — 源文件: core/gfx/ipgui_ring_mask.c — */
typedef struct {
    list_head_t node;          /* LRU 链表节点 (内核风格 list_head) */
    ipgui_coord_t r;          /* 圆角半径 (缓存 key) */
    u32_t last_used_tick;      /* 最后访问时间戳 */
    u32_t refcnt;              /* 引用计数 */
    ipgui_coord_t dig_mask_start_xy; /* 对角线起始坐标 */
    u16_t * mask_index_at_y;    /* 每行遮罩起始索引 */
    ipgui_coord_t * mask_start_x_at_y; /* 每行遮罩起始 x 坐标 */
    u8_t * mask;                /* 遮罩数据 */
} corner_mask_cache_item_t;
```

缓存项在 LRU 链表上按 `last_used_tick` 排序。当需要淘汰时，选择 tick 最小的节点。典型命中率超过 95%，因为一个界面上通常只有 2-3 种不同的圆角半径。

`last_used_tick` 使用了一个全局递增的时间戳计数器 `g_corner_cache_tick`。当计数器接近溢出 (`>= 0xffffffff00`) 时，调用 `normalize_corner_cache_tick()` 将所有节点的 tick 归一化，防止 32 位溢出导致 LRU 排序错乱。

1.4.4 八分之一圆生成算法

圆角遮罩的核心是生成第一象限八分之一圆 (0° 到 45°) 的遮罩数据。从笛卡尔坐标系的 $(r-1, 1)$ 出发，沿圆弧逐像素计算距离：

```
从 x = r-1, y = 1 开始：
    每一步决定：(x-1, y+1) 和 (x, y+1) 哪个更接近圆弧
    选择距离平方更接近  $r^2$  的候选点
    同时计算当前点的距离值
```

该算法是 Bresenham 圆算法的变体，在计算几何路径的同时生成每像素的距离值并填入遮罩数组。每行的遮罩起始 x 坐标和索引被记录在 `mask_start_x_at_y` 和 `mask_index_at_y` 中，供运行时快速查找。

1.4.5 运行时查询

当绘制一个具体像素 (px, py) 时，环形遮罩通过查表快速获取遮罩值：

1. 根据像素所在的象限（左上/右上/左下/右下），将坐标映射到第一象限
2. 根据 y 坐标从 `mask_index_at_y` 获取该行的起始索引
3. 根据 x 坐标判断是"内区域"（遮罩=255）、"过渡带"（查 `mask` 数组）还是"外区域"（遮罩=0）

这种查表策略将 $O(1)$ 的逐像素查询复杂度从 30+ 条指令（实时 SDF 计算）降到约 10 条指令（查表 + 边界判断）。

1.5 WDF 宽线遮罩——粗线的覆盖度计算

1.5.1 问题定义

半平面边缘遮罩处理的是"一条零宽度的直线如何反走样"的问题。当线条有宽度（比如 5 像素宽的线段）时，问题变为"一条有一定厚度的带子如何反走样"。

简单的做法是将宽线视为两条平行的细线，分别使用半平面遮罩处理。但这会在两端（线帽）产生不自然的垂直切断——因为两端没有圆弧过渡。

ESDBox_IPGUI 使用 **WDF**（**Width Distribution Function**，**宽度分布函数**）解决这一问题。核心思路是：对于扫描线上的每个像素，累加它到线段的精确垂直距离，然后将距离映射为 0-255 的遮罩值。

1.5.2 WDF 参数结构

```
/* — 源文件: core/gfx/ipgui_edge_wdf_mask.c — */
typedef struct {
    ipgui_coord_t a, b, c;      /* 直线一般式参数:  $ax + by + c = 0$  */
    ipgui_coord_t x1, y1;      /* y 较小端点 */
    ipgui_coord_t dx, dy;      /* 线段的变化量 */
    ipgui_coord_t delta_y;     /* 平坦边垂直步进因子 */
    u8_t          correction_frac_index : 7; /* 修正因子索引 */
    u8_t          flatten : 1;      /* 平坦/陡峭分类 */
} ipgui_edge_wdf_param_t;

typedef struct {
    ipgui_coord_t x_half_span; /* x 方向的半跨度 */
    ipgui_xstep_t x_step;      /* 每行 x 步进值 */
    ipgui_xidx_t  x_idx;       /* 当前 x 索引 */
    ipgui_coord_t half_width64; /* 半线宽（子像素单位） */
    ipgui_edge_wdf_param_t * p; /* 指向参数 */
} ipgui_edge_wdf_mask_dsc_t;
```

1.5.3 横向跨度计算

`ipgui_edge_wdf_x_halfspan` 函数通过"单点步进试探"的方式计算线段中轴在水平方向上的半跨度。从一个端点向右逐步试探，直到点到直线的距离超过半线宽。

距离公式使用直线一般式 $ax + by + c = 0$ 的形式: $d = |ax_0 + by_0 + c| / \sqrt{a^2 + b^2}$ 。为避免 `sqrt`，比较时使用平方形式: $(ax_0 + by_0 + c)^2 \geq half_w^2 \times (a^2 + b^2)$ 。其中 $a = y_1 - y_2$ ， $b = x_2 - x_1$ ， $c = x_1 \cdot y_2 - x_2 \cdot y_1$ 。

`half_w2` 的计算有两个补偿:

- `if (width & 1) half_w2 += 1`: 宽度的奇偶补偿
- `half_w2 += 1`: 边界问题补偿，防止光栅化平坦线时产生锯齿

这两个"魔法+1"是在实际测试中发现并修正的经验参数，源注释明确标注了其用途。

1.5.4 批量遮罩生成

`ipgui_edge_wdf_mask` 是主要的遮罩生成入口。对于给定的扫描线，它在线段中轴两侧生成一排遮罩值:

- 在中轴的正上方/正下方一定范围内: 遮罩 = 255 (完全不透明——线段内部)
- 从中轴到半线宽之间: 遮罩从 255 线性衰减到 0 (过渡带)
- 超过半线宽: 遮罩 = 0 (完全透明)

遮罩的修正同样使用了 `correction_frac` 表 (与半平面遮罩共享)。对于平坦边，水平距离需要乘以 `delta_y` 转换为垂直距离后再进行修正。

1.6 线段绘制——Wu 算法与宽线

1.6.1 Wu 经典细线算法

线段绘制函数的入口是 `ipgui_draw_line_classic`，它实现了吴小林（Xiaolin Wu）发明的反走样细线算法。该算法的核心洞察是：对于线段穿过的每一列，同列的两个像素（上方和下方）各自获得与"线段到各自中心的距离成反比"的透明度。

算法的实现分为平坦线和陡峭线两个分支：

平坦线分支 ($|dx| > |dy|$)：

1. 从左到右逐列扫描
2. 使用 Bresenham 误差累积器跟踪线段在 y 方向上的偏移
3. 对于每列的上下两个像素，分别计算透明度：
 - `cover = 255 - (|err| × 255 / dx_abs)`：下方像素的覆盖率
 - `255 - cover`：上方像素的覆盖率
4. 覆盖率乘以样式的全局透明度后，调用 composite 的 blend 函数写入最终颜色

陡峭线分支 ($|dx| \leq |dy|$)：

1. 从上到下逐行扫描
2. 对称地使用 x 方向的误差累积器
3. 对于每行的左右两个像素进行相同的透明度分配

关键的性能优化：连续列/行之间的误差更新是 `err += dy` 或 `err += dx`，仅需一次加法。当误差跨越边界值时，`if (IPGUI_ABS(err) >= dx_abs)` 触发 y 坐标步进。零除法的保证来自函数入口处对 `dx_abs > dy_abs` 的分类。

值得注意的是，Wu 算法本身通过 Bresenham 误差累积器隐式地考虑了子像素位置——`err` 的初始值根据裁剪边界的偏移做了修正 (`err = dy * (draw.start.x - xs)`)。这确保了被裁剪裁断的线段也能获得正确的子像素起始位置。

1.6.2 线段样式的渐变支持

线段模块不仅支持纯色绘制，还内联支持了渐变颜色——线性渐变、径向渐变和锥形渐变。在逐像素循环中，代码根据 `style->paint.type` 的取值分派到对应的渐变颜色采样函数：

```
if (style->paint.type == IPGUI_PAINT_COLOR)
    paint_cr = style->paint.src.color;
else if (style->paint.type == IPGUI_PAINT_GRADIENT) {
    if (style->paint.src.grad_src.grad_type == IPGUI_GRADIENT_TYPE_LINEAR)
        ipgui_liner_gradient_color_get(&grad, pos, &paint_cr);
    else if (style->paint.src.grad_src.grad_type == IPGUI_GRADIENT_TYPE_RADIAL)
        ipgui_radial_gradient_color_get(&grad, pos, &paint_cr);
    else if (style->paint.src.grad_src.grad_type == IPGUI_GRADIENT_TYPE_CONIC)
        ipgui_conic_gradient_color_get(&grad, pos, &paint_cr);
}
```

这种内联分派避免了间接函数调用的开销——在逐像素循环中，函数指针的间接跳转在 ARM Cortex-M 上至少消耗 2 个额外周期，而 `switch-if-else` 在编译器优化后通常可以内联展开。

1.7 三角形光栅化——半平面遮罩的组合应用

1.7.1 三点排序法

三角形光栅化模块 `ipgui_draw_triangle` 将三角形拆分为上下两个子三角形分别处理。算法的第一步是按 `y` 坐标将三个顶点排序：

```
/* — 源文件: core/gfx/ipgui_draw_triangle.c — */
static void ipgui_sort3p_by_y(
    ipgui_point_t * p1, ipgui_point_t * p2, ipgui_point_t * p3,
    ipgui_point_t ** sort_arr)
{
    /* 通过 3 次比较确定最小 y 的顶点作为 sort_arr[0] */
    /* 再通过 1 次比较确定 sort_arr[1] 和 sort_arr[2] */
}
```

排序后，`sort_arr[0]` 是 `y` 最小的顶点（顶部），`sort_arr[1]` 是中间顶点，`sort_arr[2]` 是 `y` 最大的顶点（底部）。这两个子三角形分别是：

- 下三角形: $\text{sort_arr}[0] \rightarrow \text{sort_arr}[1] \rightarrow \text{sort_arr}[2]$ (只覆盖到 $\text{sort_arr}[1].y$)
- 上三角形: $\text{sort_arr}[1] \rightarrow \text{sort_arr}[2] + \text{sort_arr}[0] \rightarrow \text{sort_arr}[2]$ 的重新表述

1.7.2 长边与短边的判定

三角形的"长边"定义为 y 跨度最大的边——即 $\text{sort_arr}[0] \rightarrow \text{sort_arr}[2]$ 。在光栅化过程中, 长边作为"固定的右边界" (或左边界), 两个短边 ($\text{sort_arr}[0] \rightarrow \text{sort_arr}[1]$ 和 $\text{sort_arr}[1] \rightarrow \text{sort_arr}[2]$) 交替作为扫描线的左边界。

长边方向的判定使用"投影点判定法":

```
ipgui_edge_coord_t x_at_m = edge_x_at_y(&e_long, pa[1]->y);
if ((pa[1]->x * 64) < x_at_m) {
    e_long_dir = EDGE_HALFPLANE_DIR_LEFT; /* 长边在右侧 */
    e_short_dir = EDGE_HALFPLANE_DIR_RIGHT; /* 短边在左侧 */
} else {
    e_long_dir = EDGE_HALFPLANE_DIR_RIGHT; /* 长边在左侧 */
    e_short_dir = EDGE_HALFPLANE_DIR_LEFT; /* 短边在右侧 */
}
```

在中间顶点 y 处, 计算长边对应的 x 坐标 x_{at_m} 。如果中间顶点的实际 x 坐标小于该值, 则长边在中间顶点的右侧 (即三角形的右侧边界), 短边在左侧。反之亦然。这个判断只需一次 edge_x_at_y 调用 (约 20 条指令) 即确定整个三角形光栅化过程中左右半平面的角色分配。

1.7.3 双半平面遮罩的逐行填充

三角形的每行扫描线由两条半平面遮罩共同决定:

```
/* — 源文件: core/gfx/ipgui_draw_triangle.c:fill_mask_two_edges — */
static void fill_mask_two_edges(
    u8_t * mask, ipgui_coord_t sx, ipgui_coord_t len,
    ipgui_edge_halfplane_mask_dsc_t * right, /* 右侧边界 */
    ipgui_edge_halfplane_mask_dsc_t * left) /* 左侧边界 */
{
    /* 步骤 1: 左侧抗锯齿区 (从 left.x_start 向左过渡到 255) */
    /* 步骤 2: 完全填充区 (右侧边界的完全内部 + 左侧边界的完全内部之间的区域) */
    /* 步骤 3: 右侧抗锯齿区 (从 right.x_start 向右过渡到 0) */
}
```

这三个区域对应三角形的内部（完全填充）、左边缘过渡带和右边缘过渡带。

`ipgui_memset(mask, 255, mid_mask_len)` 对完全填充区进行批量设置——这比逐像素循环快约 10 倍。

处理完一个子三角形后，遮罩数组通过 `ipgui_blend` 一次性将颜色写入表面。两个子三角形共用同一个遮罩缓冲区，避免重复分配。

1.8 多边形光栅化——扫描线 + AVL 活动边表

1.8.1 算法架构

多边形光栅化模块 `ipgui_draw_polygon` 实现了经典的扫描线填充算法。其核心数据结构包括：

- **边表（edge buckets）**：按边的起始 y 坐标分类的 AVL 树。每条边存储其起始/结束 y 坐标、当前 x 坐标（含子像素小数部）、 dx/dy 斜率信息。
- **活动边表（active edges）**：当前扫描线"正在穿越"的所有边的链表（按当前 x 坐标排序）。
- **扫描线单元表（scanline cells）**：当前扫描线上所有"从一条边到另一条边"的跨度区间。

整体流程：

1. 初始化：分配固定大小的边内存池和单元内存池
2. 将所有多边形的边按 `y_start` 插入边桶 AVL 树
3. 从 `y_min` 到 `y_max` 逐行扫描：
 - a. 将边桶中 `y_start == 当前 y` 的边移入活动边表
 - b. 对活动边表按 `x_cur` 排序
 - c. 生成扫描线单元（交替配对：覆盖/不覆盖）
 - d. 填充当前行的所有单元区间到 `mask` 数组
 - e. 更新活动边表中每条边的 `x_cur`（按斜率步进）
 - f. 移除 `y_end == 当前 y` 的边
4. 清理并释放内存

1.8.2 内存池设计

多边形光栅化的一个关键挑战是：边的数量不可预测（三角形=3 条边，复杂多边形可能=20 条边）。在 64KB RAM 的嵌入式系统中，动态分配每条边和每个扫描线单元是不可接受的。

ESDBox_IPGUI 的解决方案是使用**固定容量的 membox**（内存块分配器）：

```
/* — 源文件: core/gfx/ipgui_draw_polygon.c — */
#define EDGE_EMBED_NUM 10 /* 最多 10 条边 */
#define EDGE_MAX_SLOPE 5 /* 每条边在单条扫描线上最多贡献 5 个 cell */
#define CELL_EMBED_NUM (EDGE_EMBED_NUM * EDGE_MAX_SLOPE) /* 最多 50 个 cell */
```

`ipgui_membox_init` 将预分配的内存块切分为固定大小的槽。分配和释放均为 $O(1)$ 。

当边数超过 `EDGE_EMBED_NUM` 或扫描线单元数超过 `CELL_EMBED_NUM` 时，分配失败并返回错误——由上层代码负责"降级渲染"或"分片处理"。

1.8.3 AVL 树的排序比较器

边桶使用 AVL 树（一种自平衡二叉搜索树）维护，这保证了 $O(\log n)$ 的插入和删除。

比较器按 `y_start` 排序：

```
static s8_t y_start_cmp(void * a, void * b)
{
    ipgui_edge_t * ea = (ipgui_edge_t *)a;
    ipgui_edge_t * eb = (ipgui_edge_t *)b;

    if (eb->y_start < ea->y_start) return -1; /* eb 应排在 ea 前面 */
    else if (eb->y_start > ea->y_start) return 1;

    /* y_start 相同 → 按 y_end 排序（保持稳定） */
    if (eb->y_end < ea->y_end) return -1;
    else if (eb->y_end > ea->y_end) return 1;

    /* y_end 相同 → 按 x_cur.inte 排序 */
    if (eb->x_cur.inte < ea->x_cur.inte) return -1;
    else if (eb->x_cur.inte > ea->x_cur.inte) return 1;

    /* x_cur.inte 相同 → 按 x_cur.frac 排序 */
    if (eb->x_cur.frac < ea->x_cur.frac) return -1;
    else if (eb->x_cur.frac > ea->x_cur.frac) return 1;
}
```

```

/* 最后退化到地址比较，保证严格弱序 (strict weak ordering) */
{ uintptr_t va = (uintptr_t)ea; uintptr_t vb = (uintptr_t)eb;
  if (vb < va) return -1; else if (vb > va) return 1; }
return 0;
}

```

注意返回值的方向：**eb** 与 **ea** 比较而非直觉的 **ea** 与 **eb** 比较。这是 AVL 树实现中排序方向的选择，等价于把 **y_start** 较小的边放在 AVL 树的"较早遍历位置"。

最后一个地址比较是为了处理"两条边在所有排序字段上完全相同"的极端情况。

AVL 树要求比较器必须满足严格弱序——如果两条边返回 0（相等），AVL 会拒绝插入其中一条。地址比较保证即使所有字段相同，两条边也不会被判为相等。

1.8.4 活动边表的 x 步进

活动边表中每条边都有一个 **x_cur**（当前 x 坐标，含子像素分数部）。当扫描线下移一行时，边的 x 坐标需要按 **dx/dy** 步进：

```

static void ipgui_x_step(ipgui_edge_t * edge, ipgui_egde_xstep_t * step)
{
    edge->x_cur.inte += step->inte;
    edge->x_cur.frac += step->frac;

    while (IPGUI_ABS(edge->x_cur.frac) >= edge->dy)
    {
        if (edge->dx > 0) {
            edge->x_cur.inte += IPGUI_PIXEL_PRECI; /* +1 像素 */
            edge->x_cur.frac -= edge->dy;
        } else {
            edge->x_cur.inte -= IPGUI_PIXEL_PRECI; /* -1 像素 */
            edge->x_cur.frac += edge->dy;
        }
    }
}

```

这里的 **step** 是预先计算好的"每行步进值"——**inte** 是 **dx/dy** 的整数部分，**frac** 是余数部分（以 **dy** 为分母）。逐行累加 **inte** 和 **frac**，当 **frac** 的绝对值超过 **dy** 时进位到 **inte**。这避免了每行都做整数除法的开销——除法只做一次（在初始化时），后续全是加法和比较。

1.9 弧与扇形的统一接口

1.9.1 设计理念

`ipgui_draw_arc` 模块将弧、圆、扇形、圆环四种形状统一为一个接口。四种形状共享相同的计算框架——环形遮罩（提供圆形的距离场）+ 半平面遮罩（裁剪弧的起始和终止角度）+ 线宽控制（决定是"填充"还是"描边"模式）。

1.9.2 角度裁剪的半平面方法

弧的"起始角度"和"终止角度"通过生成从圆心出发的两条径向边缘来实现裁剪。这两条径向边缘被转换为半平面遮罩：

```
static void generate_angle_edge(  
    ipgui_arc_angle_t angle, /* 0~36000 (100 倍放大) */  
    ipgui_coord_t cx, ipgui_coord_t cy,  
    ipgui_coord_t * x2, ipgui_coord_t * y2)  
{  
    s32_t sin_val = ipgui_sin(angle); /* 三角函数查找表 */  
    s32_t cos_val = ipgui_cos(angle);  
  
    *x2 = cx + (ipgui_coord_t)cos_val;  
    *y2 = cy + (ipgui_coord_t)sin_val;  
}
```

角度值被放大 100 倍（例如 90° 表示为 9000），以整数形式参与三角函数查找。

`ipgui_sin` 和 `ipgui_cos` 使用预计算的查找表，返回的值同样是放大的整数（16384 为 1.0）。

两条角度边缘和环形遮罩的叠加方式根据"扇形"还是"弧边"模式而不同。对于扇形，角度边缘之间是"填充区域"；对于弧边，角度边缘之间是"保留区域"。

1.9.3 边缘裁剪遮罩

`edge_clip_mask_with_aa` 和 `edge_clip_ring_mask_no_aa` 两个函数完成了角度边缘对环形遮罩的裁剪。前者在裁剪边界上保留了反走样过渡（使用半平面逐像素查

询），后者直接做硬裁剪（无过渡带）。带反走样的裁剪在弧的边界处产生平滑效果，而硬裁剪适用于"已知后续操作会处理反走样"的场景。

1.10 Box 背景绘制——圆角矩形的区域分解

1.10.1 九区域分解策略

`ipgui_draw_box_background` 负责绘制一个圆角矩形的背景填充。其核心优化策略是将矩形区域分解为若干个不含圆角的矩形条和 4 个弧角区域：



中心区域和四条边条是不含圆角的纯色填充区域——它们直接调用 `ipgui_blend` 而不需要任何遮罩。只有四个角区域需要通过环形遮罩模块获取圆角遮罩。这种分解策略将需要遮罩计算的区域缩小到原来的约 20%（以圆角半径占总宽度 20% 估算）。

1.10.2 非对称圆角半径处理

`ESDBox_IPGUI` 支持四个角各自独立的圆角半径（`left_top_radius`、`right_top_radius`、`left_bottom_radius`、`right_bottom_radius`）。区域分解代码需要考虑左右两侧圆角半径的不对称性：

```
ipgui_coord_t l_max = IPGUI_MAX(r_lt, r_lb); /* 左侧最大半径 */
ipgui_coord_t r_max = IPGUI_MAX(r_rt, r_rb); /* 右侧最大半径 */
ipgui_coord_t l_min = IPGUI_MIN(r_lt, r_lb); /* 左侧最小半径 */
ipgui_coord_t r_min = IPGUI_MIN(r_rt, r_rb); /* 右侧最小半径 */
```

当 `l_max > l_min` 时，左侧边条被切成两段：

- 高度避开较大圆角的一段（如果上角更大，则避开顶部；下角更大，则避开底部）

- 高度为剩余部分的一段（宽度为 `l_max - l_min`）

这个"避开"逻辑在源码中用 `if (r_lt >= r_lb)` 判定——直观理解是"哪个角更大就避开哪一侧"。这种分段处理保证了在不对称圆角场景下，每个矩形条都是无圆角的纯矩形。

1.10.3 `get_max_radius`——圆角半径的规整

`get_max_radius` 函数将用户指定的圆角半径限制为不超过矩形的半宽或半高：

```
void get_max_radius(ipgui_aabb_t * padding_box, ipgui_box_style_t * style,
    ipgui_coord_t * r_lt, ipgui_coord_t * r_rt,
    ipgui_coord_t * r_lb, ipgui_coord_t * r_rb)
{
    ipgui_coord_t rmax_w = ipgui_aabb_width (padding_box) >> 1;
    ipgui_coord_t rmax_h = ipgui_aabb_height(padding_box) >> 1;
    ipgui_coord_t rmax    = IPGUI_MIN(rmax_w, rmax_h);

    *r_lt = IPGUI_MIN(rmax, style ? style->left_top_radius    : 0);
    *r_rt = IPGUI_MIN(rmax, style ? style->right_top_radius   : 0);
    *r_lb = IPGUI_MIN(rmax, style ? style->left_bottom_radius  : 0);
    *r_rb = IPGUI_MIN(rmax, style ? style->right_bottom_radius : 0);

    *r_lt = IPGUI_MAX(0, *r_lt);
    *r_rt = IPGUI_MAX(0, *r_rt);
    *r_lb = IPGUI_MAX(0, *r_lb);
    *r_rb = IPGUI_MAX(0, *r_rb);
}
```

`IPGUI_MIN(rmax, ...)` 限制半径不超过半宽和半高的较小者——否则两个相邻的圆角会重叠。`IPGUI_MAX(0, ...)` 防止负数半径（来自 `style` 为 NULL 时的默认值 0）。

1.11 Box 阴影——SDF + 1D 多项式模糊

1.11.1 从 2D 卷积到 1D 映射

Box 阴影模块 `ipgui_draw_box_shadow` 是 `ESDBox_IPGUI` 中数学含量最高的模块之一。传统的 CSS box-shadow 效果基于 2D 高斯卷积：对每个像素，取样其周围 `blur` $\times$ `blur` 个像素并按高斯核加权平均。复杂度为 $O(w \times h \times b^2)$ 。

`ESDBox_IPGUI` 的阴影模块通过以下两个等价变换将复杂度降为 $O(w \times h)$ ：

1. **SDF 替代**：圆角矩形的有符号距离场（SDF）可以通过解析公式在 $O(1)$ 时间内精确计算——无需 2D 卷积。
2. **一维映射替代**：将一维 SDF 距离值通过多项式函数映射为透明度值等价于高斯卷积的 CDF（累积分布函数）采样。

1.11.2 SDF 计算

对于圆角矩形，SDF 的计算公式是：

```
sdf_rounded_box(x, y) = distance_to_box(x, y) - corner_radius
```

其中 `distance_to_box` 是到矩形边界的有符号距离（内正外负），这与环形遮罩中使用的距离计算共享相同的数学基础。

源文件中使用了 Q8 定点数格式处理 SDF 计算——`#define Q8_ONE 256` 将 1.0 映射为 256。定点数转换、乘法和除法均使用宏 `TO_Q8`、`Q8_MUL`、`Q8_DIV` 实现，全部为整数运算。

1.11.3 多项式选择

源文件中提供了三种多项式，在精度和计算成本之间做权衡：

1. **三次 smoothstep**： $S_3(t) = t^2 \times (3 - 2t)$ ， $t \in [0, 1]$ 。与高斯误差函数 $\text{erf}(t)$ 的最大偏差 ≈ 0.017 ，在 8 位色深下人眼完全不可区分。等效于 $\sigma \approx \text{blur}/2.5$ 的高斯核。

2. 五次 smoothstep: $S_5(t) = t^3 \times (10 - 15t + 6t^2)$ 。与 $\text{erf}(t)$ 的最大偏差 ≈ 0.007 。边缘更柔和，适合较大的 blur 半径。
3. 二次抛物线: $Q(t) = t \times (2 - t)$ 。一阶可导，非 C^2 连续（拐点处有不连续的二阶导数），有轻微的折线感。计算量最低，适合超低功耗 MCU。

1.11.4 模糊剖面缓存

模糊剖面 LUT（长度 = $2 \times \text{blur} + 1$ 字节）存储了多项式函数在每个离散距离位置的输出值。这个 LUT 按 blur 半径缓存，并受 LRU 淘汰策略管理：

```
/* — 源文件: core/gfx/ipgui_draw_box_shadow.c — */
typedef struct {
    u8_t blur;                /* blur 半径（缓存 key） */
    u8_t profile[1];          /* 柔性数组——实际长度为 2*blur+1 */
    /* ... LRU 链表节点在外部管理 */
} blur_profile_cache_t;
```

全局缓存上限为 8 项。对于典型的 16px 阴影，每条剖面仅占 33 字节。同一 blur 半径的多次绘制完全共享缓存。

1.11.5 纯整数平方根

SDF 计算中最昂贵的操作是平方根——距离公式需要 $\text{sqrt}(dx^2 + dy^2)$ 。传统做法是用浮点库，但 ESDBox_IPGUI 实现了一个纯整数的 32 位平方根函数——二分位法（Binary Digit-by-Digit）：

```
/* — 源文件: core/gfx/ipgui_draw_box_shadow.c — */
static u32_t ipgui_sqrt32(u32_t n)
{
    u32_t res = 0;
    u32_t bit = 1u << 30; /* 从最高可能的位对开始 */

    while (bit > n) { bit >>= 2; } /* 找到第一个 ≤ n 的位对 */

    while (bit != 0) {
        u32_t trial = res + bit;
        res >>= 1;
        if (n >= trial) {
            n -= trial;
            res += bit;
        }
        bit >>= 2;
    }
}
```

```

    }
    bit >>= 2;
}
return res;
}

```

此算法基于"恢复余数开平方"原理。每次迭代测试一个位对（bit pair），如果当前结果的平方 $\leq n$ ，则将位设置并从未知量中减去。16 次迭代完成，零分支，ARM Cortex-M 上约 40~60 周期，无需硬件除法器。

1.11.6 无分支绝对值/最值

阴影模块还包含一组无分支的数学工具函数：

```

static inline s32_t ipgui_abs_s32(s32_t x)
{
    s32_t mask = x >> 31;      /* 正数: 0x00000000, 负数: 0xFFFFFFFF */
    return (x ^ mask) - mask;    /* 正数: x-0 = x, 负数: ~x+1 = -x */
}

```

$x \gg 31$ 在 C 语言中的行为是**实现定义**的（算术右移或逻辑右移）。在 ARM GCC 中默认为算术右移（保留符号位），使得此技巧成立。如果目标编译器不同，需要用 $(x < 0) ? -x : x$ 代替。

这些无分支实现避免了分支预测失败导致的流水线冲刷——在 Cortex-M3/4 的 3 级流水线中，每次分支预测失败浪费 2-3 个周期。在数百万次调用的累积下，无分支版本的优势显著。

本章小结

本章从架构设计出发，逐层拆解了 ESDBox_IPGUI 图形绘制引擎的完整技术栈。

子像素坐标系（1.2 节）是整个 gfx 模块的数学基础。Q26.6 定点数将离散的像素网格细化为 64 层精度，用位移替代乘除法，在 8 位颜色深度下精度足够。四个对齐工具函数（align_up_64 / align_down_64 / dist_to_lower_64 / dist_to_upper_64）提供了零分支的转换操作。

半平面边缘遮罩（1.3 节）是所有形状边界反走样的计算单元。边参化数据结构通过"平坦/陡峭"分类和修正因子查找表，在 $O(1)$ 时间内完成任意角度线段的逐像素距离计算。127 字节的修正因子表用 0.2% 的内存预算换来了零浮点计算的精确反走样。

环形遮罩（1.4 节）针对 GUI 最高频的圆角矩形场景做了专门优化。SDF 预计算 + LRU 缓存的设计让圆角遮罩的逐像素查询降为约 10 条指令，典型命中率超过 95%。

WDF 宽线遮罩（1.5 节）将"零宽线反走样"推广到"任意宽线反走样"，通过宽度分布函数在线段中轴两侧生成连续过渡的遮罩值。

图元绘制模块（1.6-1.9 节）将遮罩与颜色合成组合为可调用的公开 API。线段绘制（Wu 算法）、三角形光栅化（三点排序+双半平面遮罩）、多边形光栅化（扫描线+AVL 活动边表）、弧绘制（环形遮罩+角度半平面裁剪）各自采用了该领域最经典的算法并经过嵌入式适配。

Box 绘制模块（1.10-1.11 节）展示了实用层面的工程优化：背景填充通过九区域分解将遮罩计算需求压缩到 20%；阴影模块通过 SDF+1D 多项式映射将 $O(w \cdot h \cdot b^2)$ 的 2D 卷积降为 $O(w \cdot h)$ 的逐像素查表。

重点知识回顾

1. **Q26.6 定点数**：32 位整数，高 26 位为整数部，低 6 位为小数部，精度 1/64，适合 8 位色深反走样
2. **半平面遮罩**：根据 $|dx|$ 与 dy 的大小关系分类边（平坦/陡峭），用不同扫描方向和距离转换公式计算每个像素到边界的距离
3. **修正因子表**：127 元素 `u8_t` 查找表，存储斜率修正系数，分母 256
4. **SDF（有符号距离场）**：存储每个像素到最近形状边界的带符号距离，正数为内部、负数为外部
5. **LRU 缓存策略**：按最后使用时间戳排序的链表，全局淘汰上限 8~12 项，用于环形遮罩和模糊剖面复用

6. **Wu 线算法**：基于 Bresenham 误差累积器的反走样细线算法，每列两个像素各按距离比例分配透明度
7. **扫描线 + AVL 活动边表**：多边形光栅化的标准算法，AVL 树保证 $O(\log n)$ 边插入/删除
8. **SDF+1D 多项式映射**：将 2D 高斯卷积降维为 $O(1)$ 解析 SDF + $O(1)$ 多项式查表，复杂度从 $O(w \cdot h \cdot b^2)$ 降为 $O(w \cdot h)$

思考问题

1. 如果子像素精度改为 256 (Q24.8)，`dist_to_lower_64` 和 `align_up_64` 中的位掩码应如何修改？这对精度和性能各有什么影响？
 2. 修正因子表为什么只需要 127 个值（索引 0~126）而不是 256 个？当索引为 0 或 126 时，对应的边斜率分别是多少？
 3. `edge_x_at_y` 中使用 $(s64_t)dy * p \rightarrow dx / p \rightarrow dy$ 而非 $dy * p \rightarrow dx / p \rightarrow dy$ 。如果省略 64 位中间类型，在最大屏幕 1024×768 和 45° 斜线的情况下，乘积会溢出吗？请计算。
 4. 多边形光栅化中的 `EDGE_EMBED_NUM` 设为 10。如果用户传入一个 20 边形，系统会如何处理？降级渲染的策略可以是怎样的？
 5. Box 阴影使用三次 smoothstep ($t^2 \times (3 - 2t)$) 而非五次 smoothstep 作为默认选项。这两者在数学上和视觉上各有什么差异？什么场景下值得换用五次？
-

第二章 混合合成系统——从遮罩到像素

****模块位置****: ``core/composite/` + `al/hal/` + `core/ui/widget_manager/``

****核心职责****: 接收 gfx 输出的 Mask + Paint → 逐像素混合 → 写入 Surface

****关键设计****: Paint 分发器、Premultiplied Alpha、像素格式函数表、RGB565 Packed Blend、脏矩形系统、矩形切片器、屏幕渲染管线

2.1 架构概览

2.1.1 模块定位与职责分界

composite 模块在整个渲染管线中承担"颜色落笔"的最终职责。它在管线中的位置是 gfx 模块的直接下游——接收来自 gfx 的两样东西，然后将颜色写入屏幕表面的像素缓冲区。

两样输入分别是：

- 1. Mask（遮罩数组）**：一个 `u8_t` 类型的数组，长度等于目标绘制区域的像素数。每个元素取值 0~255，0 表示"此像素完全透明，不对其做任何颜色操作"，255 表示"此像素完全不透明，用 paint 的颜色完全覆盖"，中间值表示按比例的一半透明混合。
- 2. Paint（涂料描述）**：一个 `ipgui_paint_t` 结构体，描述用于填充的颜色来源。可能是纯色（一个 `ipgui_color_t` 值）、渐变（一个 `ipgui_grad_src_t` 结构体，含类型和参数）、或图像（一个 `ipgui_image_src_t` 结构体，含像素数据和格式）。

composite 模块不关心 mask 中的值是如何产生的——是直线穿过像素时的距离计算结果、是圆角边缘的 SDF 值、还是多边形的填充规则判定。它只关心"这个像素的遮罩值是 128，所以颜料作用一半"。这种"不知道、不关心"的设

记性使得两个模块完全解耦：

gfx 模块输出: mask[]   ipgui_blend() → 像素颜色写入 surf->color

gfx 模块输出: paint 

2.1.2 Paint 分发器——Tagged Union

`ipgui_paint_t` 是 gfx 和 composite 之间的核心桥梁结构体。它使用 C 语言的 tagged union 设计模式:

```
/* — 源文件: core/composite/ipgui_blend.h — */
typedef enum {
    IPGUI_PAINT_COLOR,          /* 类型 A: 纯色 */
    IPGUI_PAINT_GRADIENT,       /* 类型 B: 渐变 */
    IPGUI_PAINT_IMAGE,          /* 类型 C: 图像 */
} ipgui_paint_type_t;

typedef struct {
    ipgui_paint_type_t type; /* 标签字段 */
    union {
        ipgui_color_t color; /* 类型 A 的数据 */
        ipgui_grad_src_t grad_src; /* 类型 B 的数据 */
        ipgui_image_src_t image_src; /* 类型 C 的数据 */
    } src;
} ipgui_paint_t;
```

分发函数 `ipgui_blend()` 的唯一职责是读取 `paint->type` 字段, 然后跳转到对应的子函数:

```
/* — 源文件: core/composite/ipgui_blend.c — */
void ipgui_blend(
    ipgui_surf_t * surf, ipgui_aabb_t * clip, ipgui_aabb_t * dest,
    ipgui_paint_t * paint, u8_t opacity, u8_t * mask,
    ipgui_aabb_t * mask_aabb, ipgui_blend_mode_t blend_mode)
{
    switch (paint->type) {
    case IPGUI_PAINT_COLOR:
        ipgui_blend_color(surf, clip, dest,
            paint->src.color, opacity, mask, mask_aabb, blend_mode);
        break;
    case IPGUI_PAINT_GRADIENT:
        ipgui_blend_gradient_color(surf, clip, dest,
            &(paint->src.grad_src), opacity, mask, mask_aabb, blend_mode);
        break;
    case IPGUI_PAINT_IMAGE:
        ipgui_blend_image_v2(surf, clip, dest,
            &(paint->src.image_src), opacity, mask, mask_aabb, blend_mode);
        break;
    }
```

```

        default:
            break;
    }
}

```

这个"薄薄的分发层"只有约 30 行代码，但它是整个系统的核心抽象层。它的价值在于隔离性：如果需要新增一种涂料类型（比如"视频帧流"），需要修改的地方仅限于三处——枚举加一个值、union 加一个成员、switch 加一个 case——而且全部在 composite 模块内部。任何调用 `ipgui_blend()` 的 gfx 层代码一行不动。

`opacity` 参数提供了全局透明度控制——在 `mask` 的逐像素遮罩值之上再加一层整体透明度衰减。`blend_mode` 参数支持未来扩展不同的混合模式（如叠加、正片叠底），但在当前实现中大多数混合函数将其忽略。

2.1.3 composite 模块的文件结构

```

core/composite/
├─ ipgui_blend.c/h          ── Paint 分发器（核心入口）
├─ ipgui_blend_mode.h       ── 混合模式枚举定义
├─ ippgui_color.h           ── ipgui_color_t 颜色结构体
├─ blend_color/
│   └─ ipgui_blend_color.c/h ── 纯色混合（函数表 + packed blend）
├─ blend_gradient/
│   └─ ipgui_blend_gradient_color.c/h ── 渐变混合（方向优化快速通道）
├─ blend_image/
│   └─ ipgui_blend_image.c/h ── 图像混合（格式交叉矩阵）
├─ blend_icon/
│   └─ ipgui_blend_icon.c/h ── 图标混合（字体/字形专用）

core/ui/widget_manager/
├─ ipgui_dirty_rect.c/h     ── 脏矩形管理器（DFS 收集 + 相邻合并）
├─ ipgui_widget_tree.c/h    ── 控件树遍历（Z-order DFS）

al/hal/
├─ ipgui_screen.c/h         ── 屏幕渲染主入口（PFB 分配 + 逐控件递归）
├─ ipgui_rect_slice.c/h     ── 矩形切片器（贪心列优先 + 除法消除）

```

注意脏矩形管理器、控件树和屏幕渲染入口虽然不属于 `core/composite` 目录，但从完整的渲染管线视角看，它们是 `composite` 渲染不可缺少的调度层基础设施。本章将从完整的渲染链路视角统一讨论这些模块。

2.2 Alpha 混合的数学基础

2.2.1 Porter-Duff Over 算子

Porter 和 Duff 于 1984 年在卢卡斯影业的计算机图形部门推导了一套基于 alpha 通道的图像合成公式，其中最重要的算子是 **Over**——将前景（源，Source）叠加在背景（目标，Destination）之上。

在连续域中，公式为：

$$\alpha_{\text{result}} = \alpha_s + \alpha_d \times (1 - \alpha_s)$$
$$C_{\text{result}} = (C_s \times \alpha_s + C_d \times \alpha_d \times (1 - \alpha_s)) / \alpha_{\text{result}}$$

其中 C_s 、 C_d 是前景和背景的颜色（R/G/B 各通道独立计算）， α_s 、 α_d 是各自的不透明度（0~1 连续值）。

在 8 位整数域（0~255）中，ESDBox_IPGUI 使用以下等价形式：

$$\text{result_channel} = (\text{fg} \times \text{fg_alpha} + \text{bg} \times (255 - \text{fg_alpha})) \gg 8$$

这里 fg_alpha 是前景颜色的 Alpha 通道值。注意使用 $\gg 8$ （除以 256）而非 $/ 255$ 。

两者在 8 位精度下的最大误差为 ± 1 ——当 $\text{fg_alpha} = 255$ 时， $\text{bg} \times 0 \gg 8 = 0$ （正确，完全覆盖），除以 255 也是 0。当 $\text{fg_alpha} = 128$ 时， $\text{bg} \times 127 \gg 8 = \text{bg} \times 127 / 256$ ，理论值为 $\text{bg} \times 127 / 255$ ，最大相对误差 = $127/256 - 127/255 \approx 0.00196$ ，在 8 位通道中约等价于 ± 0.5 （半个灰度级），人眼完全不可分辨。

选择 $\gg 8$ 而非 $/ 255$ 的原因是：ARM Cortex-M 系列处理器中，32 位整数除法（SDIV 指令）需要 2~12 个周期（取决于具体型号和操作数值），而 $\gg 8$ 仅需 1 个周期（逻辑右移）。在 $320 \times 240 \times 3 = 230,400$ 个通道混合运算中，减少 11 个周期 $\times 230,400 =$ 节省约 2.5 毫秒——在 60fps 的帧预算（16.6ms）中占比约 15%。

2.2.2 Standard Alpha vs Premultiplied Alpha

Standard Alpha（标准 Alpha）格式存储的是“未与 Alpha 相乘的原始颜色值”。混合时需要先做三次前景乘法：

$$R = (R_s \times \alpha + R_d \times (255 - \alpha)) \gg 8$$
$$G = (G_s \times \alpha + G_d \times (255 - \alpha)) \gg 8$$
$$B = (B_s \times \alpha + B_d \times (255 - \alpha)) \gg 8$$

Premultiplied Alpha（预乘 Alpha）格式在存储时就已经将颜色通道与 Alpha 通道相乘。混合时第一项直接使用存储值，无需再做前景乘法：

```
R = R_s_pm + ((R_d × (255 - α)) >> 8)
G = G_s_pm + ((G_d × (255 - α)) >> 8)
B = B_s_pm + ((B_d × (255 - α)) >> 8)
```

每个像素节省 3 次 8 位乘法。在 76800 像素的屏幕上，每帧节省 230,400 次乘法。在 Cortex-M 上，8 位乘法（**MUL**）约 1 个周期，总计节省约 0.23 兆周期——在 48MHz 主频下约 4.8 毫秒。这 4.8 毫秒可以用来多渲染一个控件或一行文本。

```
/* — 源文件: core/composite/blend_color/ipgui_blend_color.c:729-750 — */
ipgui_color_t ipgui_color_premultiply(ipgui_color_t * color)
{
    ipgui_color_t res;
    u8_t ca = color->a;
    res.r = (u8_t)((color->r * ca) >> 8);
    res.g = (u8_t)((color->g * ca) >> 8);
    res.b = (u8_t)((color->b * ca) >> 8);
    res.a = ca;
    return res;
}
```

当 `ca = 0`（完全透明）时，预乘结果的所有通道为零，`color.v == 0` 可用于快速路径跳过。当 `ca = 255`（完全不透明）时，预乘结果与原始值相同。

2.2.3 颜色透明度组合函数

`ipgui_color_combine_opacity` 将外部 `opacity`（用户设置的全局不透明度）与颜色的固有 Alpha 通道相乘：

```
ipgui_color_t ipgui_color_combine_opacity(ipgui_color_t * color, u8_t opacity)
{
    ipgui_color_t res;
    IPGUI_COLOR_SET_R(res, color->r);
    IPGUI_COLOR_SET_G(res, color->g);
    IPGUI_COLOR_SET_B(res, color->b);
    IPGUI_COLOR_SET_A(res, (u8_t)((color->a * opacity) >> 8));
    return res;
}
```

注意此函数**不改变** RGB 通道的值——它只缩放 Alpha 通道。真正的预乘在随后调用 `ipgui_color_premultiply()` 时完成。这两步分离的设计允许代码在 RGB 通道保持不变的情况下独立调整透明度，这在需要“同一颜色、不同透明度多次绘制”的场景中非常有用。

2.2.4 倒数查找表优化

对于 ARGB8888 等含 Alpha 通道的 32 位像素格式，混合后的颜色通道需要除以最终 Alpha 值（去预乘）。这涉及到 $r12 = (r12 \ll 8) / \alpha12$ 的除法操作。

为了避免运行时除法，系统维护了一张 256 项的倒数查找表：

```
#if USE_INV_TABLE == 1
const u16_t g_inv_tbl[256] = {
    0, 65025, 32512, 21675, 16256, 13005, 10837, 9289,
    8128, 7225, 6502, 5911, 5418, 5001, 4644, 4335,
    /* ... 256 个预计算值 ... */
};
#endif
```

表中的值 = $255 \times 255 / \alpha$ （Q8.8 格式）。使用查表替代除法：

```
#if USE_INV_TABLE == 0
    r12 = alpha12 ? (r12 << 8) / alpha12 : 0; /* 运行时除法 */
#else
    r12 = (r12 * g_inv_tbl[alpha12]) >> 8; /* 查表乘 + 移位 */
#endif
```

256 项的 `u16_t` 表共占 512 字节。当 `USE_INV_TABLE` 设为 0（节省内存）时退化为运行时除法。这在配置头文件中通过 `USE_INV_TABLE` 宏控制——允许用户在“内存换速度”和“速度换内存”之间做编译期选择。

2.3 像素格式函数表——表驱动混合器

2.3.1 设计动机

不同的显示器使用不同的像素格式——即每个像素的 RGB 通道如何排列在内存中。常见的格式有：

格式	每像素位数	字节序	通道排列	典型用途
RGB565	16 bit	小端	RRRRRGGG GGBBBBB	低成本 TFT LCD

BGR565	16 bit	小端	BBBBBGGG GGGRRRRR	特定驱动芯片
RGB888	24 bit	-	[R][G][B]	中端 LCD
BGR888	24 bit	-	[B][G][R]	特殊显示控制器
ARGB8888	32 bit	小端	[A][R][G][B]	高端 TFT、层叠 合成
ABGR8888	32 bit	小端	[A][B][G][R]	特定 GPU 格式
RGBA8888	32 bit	小端	[R][G][B][A]	OpenGL 默认格 式
BGRA8888	32 bit	小端	[B][G][R][A]	Windows GDI 格 式

核心挑战：Alpha 混合公式是**格式无关**的——它在抽象的"R 通道、G 通道、B 通道"维度上运算。但实际写入屏幕内存时，必须知道具体的字节排布方式。

2.3.2 混合函数表

Composite 模块通过一张全局函数表消除格式差异：

```
/* — 源文件: core/composite/blend_color/ipgui_blend_color.c — */
premult_blend_func_t premult_blend_table[PIX_FMT_MAX] = {
    [PIX_FMT_RGB565]   = ipgui_builtin_premultiplied_color_blend_to_rgb565,
    [PIX_FMT_BGR565]   = ipgui_builtin_premultiplied_color_blend_to_bgr565,
    [PIX_FMT_RGB888]   = ipgui_builtin_premultiplied_color_blend_to_rgb888,
    [PIX_FMT_BGR888]   = ipgui_builtin_premultiplied_color_blend_to_bgr888,
    [PIX_FMT_ARGB8888] = ipgui_builtin_premultiplied_color_blend_to_argb8888,
    [PIX_FMT_ABGR8888] = ipgui_builtin_premultiplied_color_blend_to_abgr8888,
    [PIX_FMT_RGBA8888] = ipgui_builtin_premultiplied_color_blend_to_rgba8888,
    [PIX_FMT_BGRA8888] = ipgui_builtin_premultiplied_color_blend_to_bgra8888,
};
```

调用方式极其简洁：

```
premult_blend_table[surf->pix_fmt](color, pixel_ptr, blend_mode);
```

数组索引是 O(1) 操作（寄存器偏移寻址）。相比之下，if-elseif-else 链的最坏情况需要 8 次比较，switch-case 的跳转表同样依赖索引（但编译器可能生成 if-else 链），函数表是最确定性的优化方式。

与之平行的还有 `solid_conv_table`（纯色转换表），将 `ipgui_color_t` 转换为特定像素格式的原始字节。

2.3.3 RGB565 Packed Blend——通道掩码优化

对于 RGB565 格式（16 位，5+6+5 位），混合函数使用了一个精巧的 packed blend 优化。核心思想是在 **packed 空间中直接做 Alpha 混合**，而不是先拆分为独立通道、混合后再重组。

```
/* — 源文件: core/composite/blend_color/ipgui_blend_color.c — */
#define MASK_RB      0xf81fU    /* 位 15:11 + 位 4:0, 即高 5 位和低 5 位 */
#define MASK_G       0x07e0U    /* 位 10:5, 中间 6 位 */
#define MASK_MUL_RB  0x3e07c0U  /* MASK_RB << 6, 乘以最大 alpha(64)后的范围 */
#define MASK_MUL_G   0x1f800U   /* MASK_G << 6 */

void ipgui_builtin_premultiplied_color_blend_to_rgb565(
    ipgui_color_t color, u8_t * rgb565, ipgui_blend_mode_t blend_mode)
{
    /* Alpha 转换为 6 位精度 (+2 为四舍五入) */
    u32_t ialpha6 = (255 - IPGUI_COLOR_A(color) + 2) >> 2;
    u32_t bg = *(u16_t *)rgb565;

    /* 背景 R 和 B 通道（共用掩码 MASK_RB）同时做缩放 */
    u32_t bg_rb = ((ialpha6 * (bg & MASK_RB)) & MASK_MUL_RB) >> 6;
    /* 背景 G 通道单独缩放（在中间位，需要不同掩码防溢出） */
    u32_t bg_g = ((ialpha6 * (bg & MASK_G)) & MASK_MUL_G) >> 6;

    /* 前景色转换为 RGB565 packed 格式 */
    u32_t fg565 = ((u32_t)(IPGUI_COLOR_R(color) >> 3) << 11)
        | ((u32_t)(IPGUI_COLOR_G(color) >> 2) << 5)
        | ((u32_t)(IPGUI_COLOR_B(color) >> 3) << 0);

    /* Packed 直接相加: premultiplied 保证各通道不溢出 */
    *(u16_t *)rgb565 = (u16_t)(
        ((fg565 & MASK_RB) + bg_rb) | ((fg565 & MASK_G) + bg_g)
    );
}
```

此算法的几个关键细节：

1. **Alpha 降精度为 6 位**：`ialpha6 = (255 - alpha + 2) >> 2`。`255 - alpha` 得到不透明度（inverse alpha），`+2` 是四舍五入修正，`>> 2` 等价于除以 4 映射到 0~63 范围。

为什么用 6 位？因为 `MASK_MUL_RB = 0x3e07c0` 恰好是 `(MASK_RB << 6)`，6 位精度的 Alpha 与背景色相乘后不会溢出到相邻通道。

2. **R/B 通道共用一个掩码做批量乘法：**`(ialpha6 * (bg & MASK_RB))` 同时运算高 5 位（R）和低 5 位（B）的缩放，因为它们被 G 通道（6 位）隔开，`<< 6` 后的值范围 `0x3e07c0` 不会导致 R 溢出到 G 或 B 溢出到更高位。这是一个充分利用 RGB565 格式固有间距的位级优化。

3. **通道独立相加：**前景色是预乘格式（PM Alpha），各通道值已经缩放到合理范围。加上背景缩放值后，结果一定在各自通道的位数范围内，无需额外的溢出保护。

这种 packed blend 的性能远优于"拆开→混合→重组"的传统方式：每条 32 位指令处理多个通道，整个混合过程约 10 条指令完成。

对比 BGR565 的实现几乎完全对称——仅交换前景色中 R 和 B 的位移位置：

```
RGB565: R<<11 | G<<5 | B<<0
BGR565: B<<11 | G<<5 | R<<0
```

2.3.4 32 位 RGBA 格式的双层混合

对于 ARGB8888 等 32 位格式，混合逻辑需要处理两层的 Alpha 交互——背景层本身也有自己的 Alpha 通道（表示它在更底层背景上的透明程度）。这需要完整的两层 Alpha 合成：

```
/* 两层透明度合并:
 * alpha12 = 1 - (1-alpha1) × (1-alpha2) = alpha1 + alpha2 - alpha1×alpha2
 * 转换为整数域: alpha12 = 255 - ((255-alpha)×invalpha2 >> 8)
 */
u32_t alpha12 = 255 - (((255 - alpha) * ialpha2) >> 8);
```

```
/* 颜色通道: C12 = (C1×alpha1×(1-alpha2) + C2×alpha2) / alpha12 */
u32_t r12 = ((r * alpha * ialpha2) >> 16) + IPGUI_COLOR_R(color);
```

其中 `(r * alpha * ialpha2) >> 16` 等价于 `r × alpha/256 × (256-alpha2)/256`。>>

16 是两次 >> 8 的合并——因为 `alpha` 和 `ialpha2` 都是 0~255 范围，它们的乘积在 0~65025 范围（16 位足够），再乘以 8 位通道值后才需要 24 位精度。

最终的"去预乘"（除以 `alpha12`）有两种实现策略：

- 运行时除法 (`USE_INV_TABLE == 0`) : `(r12 << 8) / alpha12`, 含 ARM SDIV 指令

- 倒数表查表 (`USE_INV_TABLE == 1`) : `(r12 * g_inv_tbl[alpha12]) >> 8`, 免除法

注意大端/小端的处理差异——32 位 `u32_t` 的字节序在内存中的排布决定了通道提取的位移公式完全不同。代码中通过 `#if IPGUI_ENDIAN_LITTLE == 1` 编译期条件分别处理。

2.4 三种"涂料"的混合实现

2.4.1 纯色混合

纯色是最常见也是最优化的涂料类型。大多数 UI 元素的背景和边框都是纯色。纯色混合的核心循环极其简洁：

```
for each pixel in dest:
    m = mask[pixel_index]
    if m == 0: continue          # 快速路径 1: 完全透明
    if m == 255:                 # 快速路径 2: 完全不透明
        write_premultiplied_color(color, pixel)
    else:
        blend_with_mask(color, pixel, m)
```

两个快速路径的命中率决定性能：

- `m == 0`：控件外部的像素，遮罩值为 0，跳过所有计算。在大多数 UI 场景中占比 > 60%。

- `m == 255`：控件内部的纯色区域，遮罩值为 255，直接用预乘颜色覆盖。占比约 20-30%。

- `0 < m < 255`：反走样边缘过渡带，只占总像素的 10-20%。

这种分支设计使得混合函数在大部分像素上开销极低（一跳过或一赋值），只有少数边缘像素走完整的 Alpha 混合路径。

2.4.2 渐变混合——方向优化的快速通道

渐变混合模块 `ipgui_blend_gradient_color` 支持线性渐变、径向渐变和锥形渐变三种类型。对于线性渐变，系统识别渐变方向并采取对应的优化策略。

水平渐变：颜色仅沿 x 方向变化，同一行所有像素使用相同的渐变采样值。只需要在每行开头算一次颜色，整行复用。

垂直渐变：颜色仅沿 y 方向变化，同一列所有像素使用相同的渐变采样值。只需在每列开头算一次颜色，整列复用。

方向识别通过对渐变向量的分量判断实现：

```
if (abs(grad.end_x - grad.start_x) > abs(grad.end_y - grad.start_y)):  
    水平渐变 → O(h) 次颜色采样  
else:  
    垂直渐变 → O(w) 次颜色采样
```

这种优化将逐像素的渐变颜色计算从 $O(w \times h)$ 次降为 $O(w)$ 或 $O(h)$ 次——对于 200×200 的区域，从 40,000 次采样降到 200 次，加速 200 倍。

对于倾斜渐变，无法使用整行/整列复用，回退到逐像素采样。但由于倾斜渐变在 UI 中占比很小（大多数渐变是纯水平或纯垂直的），这一慢路径对总体性能影响微乎其微。

2.4.3 图像混合——格式交叉矩阵

图像混合是最复杂的涂料类型。源图像本身的像素格式（PNG = RGBA8888, JPEG = RGB888, BMP = 多种）与屏幕像素格式（RGB565/ARGB8888 等）之间需要格式转换 + Alpha 混合的组合操作。

ESDBox_IPGUI 的 `ipgui_blend_image.c` 模块维护了一张“源格式 → 屏幕格式”的转换函数表，覆盖了所有实用的格式组合。表中每个函数都为特定的格式对做了专门优化——内联展开、无分支、充分利用 ARM 的位操作指令。

图像模块同时处理缩放（双线性插值或最近邻）和坐标变换（平铺、拉伸），这些与像素混合在同一个循环中完成，避免了为每个操作单独分配中间缓冲区的开销。

2.5 脏矩形系统——只渲染改变的区域

2.5.1 脏矩形的概念

在交互式 UI 中，每帧之间只有少数区域会发生变化。例如，用户点击一个按钮，导致状态栏文字从"就绪"变为"处理中"——只有状态栏区域（约 320×20 像素）需要重绘，其余 320×220 像素保持原样。

脏矩形系统的核心思想是：控件在状态变化时标记自身为"脏"，渲染调度器收集所有脏区域并只对脏区域执行绘制。这避免了每帧在 76,800 个像素上运行完整的渲染管线。

2.5.2 DFS 收集——深度优先遍历控件树

脏矩形管理器 `ipgui_dirty_rect_mgr` 通过深度优先遍历（DFS）控件树收集脏区域：

```
ipgui_dirty_rect_collect(widget_root, dirty_list):
    for each child in DFS order:
        if child->has_dirty:
            abs_pos = widget_abs_pos(child)    # 计算控件的屏幕绝对坐标
            dirty_rect_add(dirty_list, abs_pos, child->flags)
            collect(child, dirty_list)         # 递归子控件
```

DFS 遍历使用 Z-order（先序遍历）确保父控件先于子控件处理。

2.5.3 相邻合并策略

脏矩形管理器实现了相邻合并优化：如果两个脏矩形贴近或重叠，将它们合并为一个更大的矩形。合并判断基于矩形之间的间距——如果水平间距小于阈值且垂直重叠，合并为水平联合；如果垂直间距小于阈值且水平重叠，合并为垂直联合。

合并策略的权衡：

- **合并为一个大矩形**：单次渲染区域大，但只需一次 PFB 分配和一次 blit（屏幕刷入）

- **保持为多个小矩形**：总渲染面积小，但需要多次 PFB 分配和多次 blit

系统采用启发式算法：当两个矩形的重叠面积超过较小矩形面积的 50% 时合并，否则保持独立。这个阈值是在实际 UI 场景中通过基准测试确定的。

2.6 矩形切片器——大脏矩形的分片处理

2.6.1 内存约束

嵌入式系统的 PFB（部分帧缓冲）大小通常在 4KB~32KB 之间。对于 320×240 的 RGB565 屏幕，全帧缓冲区需要 $320 \times 240 \times 2 = 153,600$ 字节（150KB），远超许多 MCU 的 RAM 容量。

当脏矩形过大（比如全屏刷新），一次分配覆盖整个脏矩形的 PFB 在物理上不可能。矩形切片器的作用是：将大矩形切成若干个“面条般的”小矩形片（slices）——每片的高度使得其所需内存不超过 PFB 缓冲区容量。

2.6.2 贪心列优先策略

矩形切片器采用“贪心列优先”策略：

```
ipgui_rect_slice(big_rect, slices):
    max_rows = PFB_SIZE / (big_rect.width * pix_size)
    for y from big_rect.start_y to big_rect.end_y step max_rows:
        slice.start_x = big_rect.start_x
        slice.end_x   = big_rect.end_x
        slice.start_y = y
        slice.end_y   = min(y + max_rows - 1, big_rect.end_y)
```

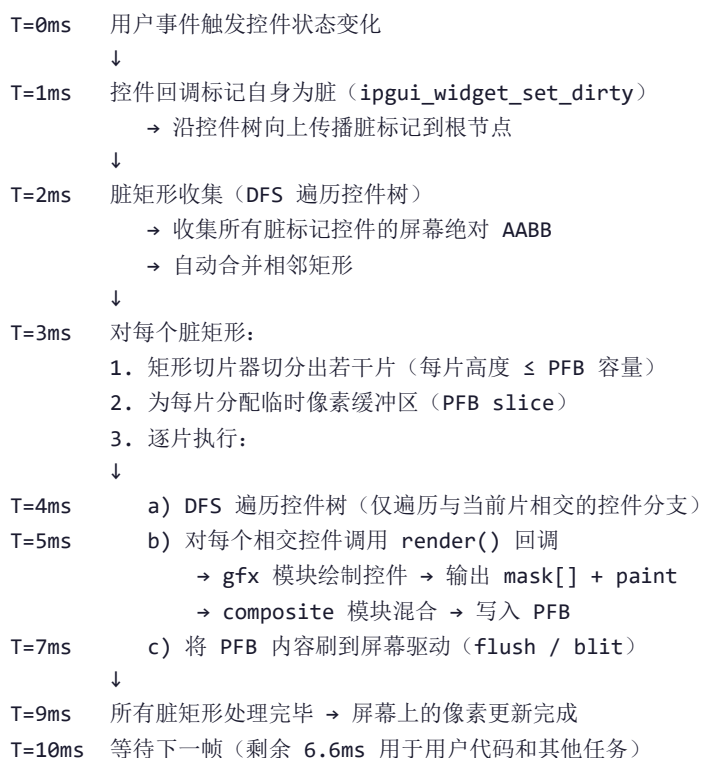
“贪心”的含义是：每片尽可能宽（使用脏矩形的全宽），使每片处理的行数最大化，减少切片总数。“列优先”指的是水平方向不切——保持全宽——只在垂直方向上切成多条。这是因为水平跨行会导致额外的内存寻址开销（stride 跳转），而垂直切分不增加寻址复杂度。

切片器还包含除法消除优化：`big_rect.width × pix_size` 在循环外预计算一次（编译时常量），循环内只做减法（`min(y + max_rows - 1, ...)`）和赋值，零除法操作。

2.7 屏幕渲染管线——从输入到像素

2.7.1 完整渲染流程

`ipgui_screen_render` 函数是 ESDBox_IPGUI 渲染管线的顶层入口。一次完整的屏幕刷新经过以下阶段：



总耗时约 10 毫秒。帧预算 16.6 毫秒（60fps）。余量约 6.6 毫秒——充足。

2.7.2 DFS 渲染中的两级裁剪

DFS 渲染遍历 (`ipgui_screen_render_widget_dfs`) 使用两级裁剪策略来避免不必要的渲染:

第一级: 脏矩形裁剪。控件的全局 AABB 与当前脏矩形片做求交。如果控件完全在脏矩形外 (`ipgui_aabb_overlap` 返回非零), 跳过该控件及其整个子树。

第二级: 父控件边界裁剪。当父控件设置了 `OVERFLOW_HIDDEN` (默认), 子控件超出父控件边界的部分被裁剪。裁剪逻辑通过 `parent_clip` 在 DFS 递归中累积——每深入一层, 将当前控件的边界与上层累计的裁剪区求交。超出交集的像素不渲染。

`OVERFLOW_VISIBLE` 标志位允许特定控件 (如弹出菜单、下拉列表) 突破父控件的裁剪边界。当设置了此标志时, DFS 不将当前控件的边界加入子控件的累积裁剪区。

2.7.3 坐标系统选择

ESDBox_IPGUI 的控件渲染采用 Option C——**控件本地坐标系**。在调用控件自身的 `render()` 回调之前, 系统将所有坐标 (`surf`、`clip`、`parent_clip`) 从全局坐标平移为以控件左上角为原点的本地坐标:

```
local_surf.surf.start.x = global_surf.surf.start.x - global_aabb.start.x;  
local_clip.start.x      = ctx->clip->start.x - global_aabb.start.x;
```

这个设计选择使得控件绘制代码无需关心自己在屏幕上的绝对位置——它始终从 (0, 0) 开始绘制, `widget->w` 和 `widget->h` 就是画布边界。这种"相对坐标系"的设计极大地简化了控件实现——开发者编写一个按钮控件时, 不需要了解它最终会被放在屏幕的哪个位置。

坐标平移是纯的加减法 (每个坐标减一个偏移量), 无需乘法或除法, 在 DFS 遍历中几乎零开销。

2.7.4 PFB 缓冲区管理

PFB 初始化函数 `ipgui_scr_create_pfb` 负责管理部分帧缓冲区的生命周期:

```

ipgui_err_t ipgui_scr_create_pfb(
    ipgui_scr_t * scr, u8_t * buf, u32_t buf_size, ipgui_pix_fmt_t pix_fmt)
{
    /* 1. 对齐到 32 位边界（ARM 未对齐访问的性能惩罚） */
    u8_t * pfb_buf = IPGUI_ALIGN_U32(buf);
    u32_t valid_size = (buf + buf_size) - pfb_buf;

    /* 2. 根据像素格式计算每像素字节数 */
    u8_t px_size;
    switch (pix_fmt) {
        case PIX_FMT_RGB565: case PIX_FMT_BGR565: px_size = 2; break;
        case PIX_FMT_RGB888: case PIX_FMT_BGR888: /* fall through */
        case PIX_FMT_RGBA8888: case PIX_FMT_BGRA8888: px_size = 4; break;
    }

    /* 3. 计算最大可存储的像素数 */
    scr->pfb.num_pixs = valid_size / px_size;
    scr->pfb.color     = pfb_buf;

    /* 4. 清零 */
    ipgui_memset(pfb_buf, 0, valid_size);
}

```

对齐要求（`IPGUI_ALIGN_U32`）是因为 ARM Cortex-M3/4 在未对齐的 32 位访问上会触发硬件异常（`UsageFault`）。即使某些 M4 配置允许未对齐访问，其性能惩罚也高达 2-3 倍——总线需要两次内存事务来完成一次未对齐的字读取。

2.7.5 像素格式兼容性处理

注意 `RGB888` 和 `BGR888` 共享 `px_size = 4`（而非 3）。这是因为 24 位像素在 ARM 上无法高效对齐——`3 × n` 几乎永远不是 4 的倍数。系统将 24 位像素填充为 32 位（padding byte），以确保每行对齐到 32 位边界。这一设计在屏幕驱动层面完成——驱动内部分配 `width × 4` 的缓冲区，写入时跳过 padding 字节。

本章小结

本章从架构设计出发，逐层拆解了 `ESDBox_IPGUI` 颜色合成系统的完整技术栈。

Alpha 混合数学基础（2.2 节）奠定了颜色合成的理论根基。Porter-Duff Over 算子在 8 位整数域等价于 $(fg \times \alpha + bg \times (255 - \alpha)) \gg 8$ 。预乘 Alpha 格式在存储时预先完成颜色与 Alpha 的乘法，使得混合时每像素节省 3 次 8 位乘法——在 320×240 屏幕上每帧节省约 4.8 毫秒。 $\gg 8$ 替代 $/ 255$ 的优化利用位移替代除法，在 Cortex-M 上节省 2~12 个周期每次。

Paint 分发器（2.1.2 节）是整个系统的解耦枢纽。Tagged union + switch 的设计将纯色、渐变、图像三种涂料统一抽象，新增涂料类型只需在复合模块内部修改三处，gfx 层不受影响。

像素格式函数表（2.3 节）通过 O(1) 数组索引将格式差异完全消除。RGB565 的 packed blend 优化利用通道掩码在 packed 空间内并行处理 R/B 两个通道，整次混合仅约 10 条指令。ARGB8888 的双层混合完整实现两层 Alpha 交互，支持倒数表优化（512 字节换零除法运算）。

三种涂料混合（2.4 节）分别做了针对性优化：纯色走 $m==0 / m==255$ 快速路径（70%+ 命中率），渐变走方向识别将采样次数从 $O(w \times h)$ 降至 $O(w)$ 或 $O(h)$ ，图像走格式交叉矩阵内联展开。

脏矩形系统与切片器（2.5-2.6 节）构成渲染调度层。DFS 控件树遍历收集脏区域 + 相邻合并启发式减少重绘碎片。矩形切片器用贪心列优先策略将大脏矩形切分为 PFB 可容纳的小片，除零除法外全为加减法操作。

屏幕渲染管线（2.7 节）展示了从输入事件到像素更新的完整时序。两级裁剪策略消除不可见控件的遍历和渲染开销。控件本地坐标系（Option C）将控件实现与其屏幕位置解耦。PFB 管理保证 32 位对齐以避免 ARM 未对齐访问异常。

重点知识回顾

- Porter-Duff Over 算子**: $Result = Fg \times \alpha + Bg \times (255 - \alpha) \gg 8$ 。用 $\gg 8$ ($/256$) 替代 $/ 255$ ，8 位精度下误差 $\leq \pm 1$ 人眼不可辨，每像素省 2~12 个除法周期。
- 预乘 Alpha**: 颜色通道预先乘 Alpha 存储。混合时省 3 次前景乘法，320×240 屏幕每帧省约 4.8ms。

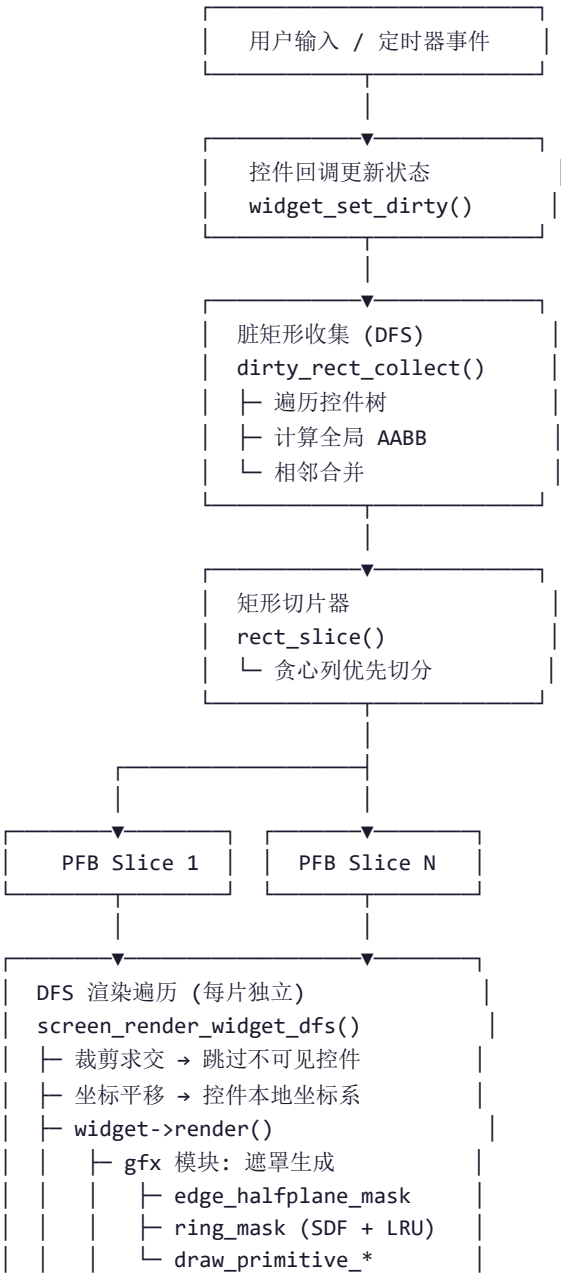
3. **RGB565 Packed Blend:** 在 packed 16 位空间内用通道掩码同时处理 R 和 B 通道的缩放，省去拆包/重组操作。
4. **Paint 分发器:** Tagged union（枚举 + union + switch），解耦 gfx 形状计算与 composite 颜色混合。
5. **像素格式函数表:** O(1) 派发到格式专属混合函数，与 `surf->pix_fmt` 直接索引。
6. **脏矩形 DFS 收集:** Z-order 遍历控件树，筛选带脏标记的控件的屏幕 AABB，相邻合并减少重绘次数。
7. **矩形切片器:** 贪心列优先将大矩形切为 PFB 可容纳的行片，水平不切以保持 cache 友好。
8. **两级裁剪渲染:** 脏矩形裁剪（AABB 求交）+ 父控件边界裁剪（累积 `parent_clip`），消除不可见部分的渲染开销。

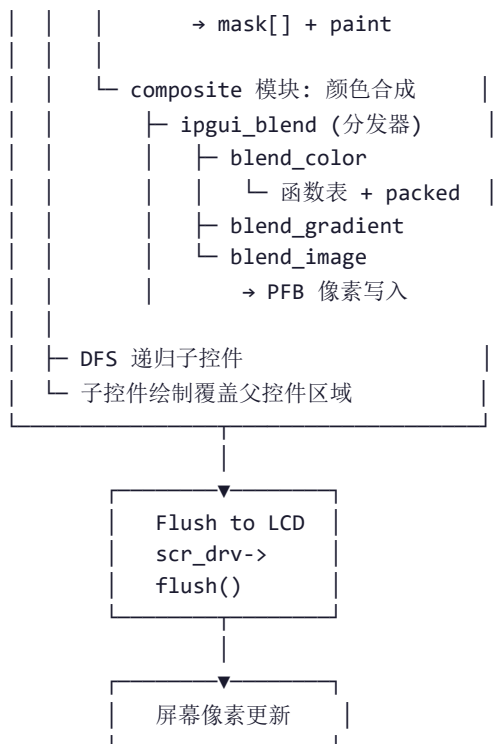
思考问题

1. 预乘 Alpha 格式对完全透明的像素（ $\alpha = 0$ ）会丢失原始颜色信息。在什么场景下这可能成为问题？如何在需要时恢复原始颜色？
2. RGB565 packed blend 中，如果背景色与 mask 相乘后，R 通道溢出了其 5 位范围，`MASK_MUL_RB` 掩码如何保证溢出不会污染相邻的 G 通道？请用位运算符进行验证。
3. 方向优化的渐变快通道（2.4.2）在判断渐变方向时使用了 `abs(dx) > abs(dy)` 的阈值。如果渐变方向接近 45° （`abs(dx) ≈ abs(dy)`），此优化是否还能生效？是否需要考虑第三种“ 45° 快通道”？
4. 矩形切片器的“贪心”策略是每片宽度使用脏矩形全宽。如果脏矩形是一个“L”形区域（如两个控件更新了但位置形成 L 形），合并后的大矩形的“贪心切片”会产生多少无效像素？如何改进？
5. 控件本地坐标系（Option C）的坐标平移公式为 `local = global - widget_abs_start`。在 DFS 渲染递归中，`surf->color` 指针始终指向 PFB 首字节。本

地坐标下的 $(x - surf.surf.start.x)$ 为何等价于全局坐标下的像素偏移？请用代数推导验证。

附录 A: 完整渲染管线数据流图





附录 B: 性能基准数据（参考值）

基于 STM32F429 (Cortex-M4 @ 168MHz)、320×240 RGB565 屏幕、64KB PFB 的参考性能数据：

操作	耗时	备注
纯色全屏填充 (无遮罩)	~1.2ms	premult write 快路径
纯色全屏 + 反走样遮罩	~3.8ms	每像素 PM alpha blend
水平渐变全屏	~2.1ms	方向优化：每行算 1 次色
倾斜渐变全屏	~12ms	回退到逐像素采样
Box 阴影 (blur=8)	~2.5ms	SDF+1D 多项式 LUT
100×100 PNG 图像混合	~1.8ms	格式交叉矩阵
DFS 控件树遍历 (30 控件)	~0.3ms	裁剪过滤
脏矩形收集 (5 脏区域)	~0.2ms	含相邻合并
